

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Telekomunikacji

Praca dyplomowa magisterska

na kierunku Inżynieria Internetu Rzeczy

Projekt i implementacja asystenta głosowego wykorzystującego lokalnie
działające modele LLM na platformie sprzętowej o ograniczonych
zasobach.

Krzysztof Kulma

Numer albumu 318476

promotor

dr inż. Daniel Paczesny

WARSZAWA 2026

Projekt i implementacja asystenta głosowego wykorzystującego lokalnie działające modele LLM na platformie sprzętowej o ograniczonych zasobach.

Streszczenie. Celem pracy było zwiększenie interaktywności wczasowiczów przebywających w domkach wypoczynkowych w Wildzie poprzez wdrożenie lokalnego asystenta głosowego, umożliwiającego osobom nietechnicznym dostęp do informacji zbieranych przez czujniki IoT. Praca obejmuje zarówno część implementacyjną — zaprojektowanie i uruchomienie systemu opartego na architekturze dwumodelowej- jak i badawczą, polegającą na empirycznym doborze optymalnego modelu językowego oraz parametrów generacji dla środowiska o ograniczonych zasobach obliczeniowych. Pracę zakończono analizą wyników i sformułowaniem wniosków dotyczących działania modeli językowych na platformach klasy Raspberry Pi.

Słowa kluczowe: asystent głosowy, large language model, smart home, inferencja lokalna, sprzęt o ograniczonych zasobach

Design and implementation of a voice assistant using locally running LLM models on a resource-constrained hardware platform.

Abstract. The aim of this project was to increase interactivity for holidaymakers staying in Wilga's holiday cottages by implementing a local voice assistant. This allows non-technical users to access information collected by IoT sensors. The project comprised an implementation component involving the design and deployment of a system based on a dual-model architecture, as well as a research component involving the empirical selection of optimal language models and generation parameters for environments with limited computational resources. The thesis concludes with an analysis of the results and conclusions regarding the performance of language models on Raspberry Pi-class platforms.

Keywords: voice assistant, large language model, smart home, on-device inference, resource-constrained devices

Spis treści

1. Wstęp	9
1.1. Tło problemu i cel pracy	9
1.2. Teza badawcza	10
1.3. Zakres pracy	10
2. Przegląd literatury i analiza rozwiązań	12
2.1. Architektury hybrydowe Edge–Cloud	12
2.2. Kwantyzacja dla inferencji na sprzęcie bez GPU	13
2.3. Wielomodelowe systemy LLM - podział ról	14
2.4. Inżynieria promptów i dostarczanie kontekstu	15
2.5. Optymalizacja hiperparametrów generacji	16
2.6. Fine tuning modeli językowych	17
2.7. Interfejsy głosowe: rozpoznawanie mowy, detekcja aktywności głosowej i synteza mowy	17
3. Koncepcja i architektura systemu	19
3.1. Założenia projektowe	19
3.2. Architektura Nauczyciel–Uczeń (Teacher–Student)	19
3.3. Schemat przepływu danych w systemie	20
4. Środowisko projektowo-badawcze	23
4.1. Wybór platformy brzegowej	23
4.2. Architektura sprzętowa	23
4.3. Modele LLM w edge computing’u	24
4.4. Wykorzystane oprogramowanie i technologie	25
4.4.1. Aplikacja użytkownika	25
4.4.2. Warstwa serwerowa i komunikacyjna	25
4.4.3. Moduły STT i TTS	25
4.4.4. LLM	27
4.4.5. Context Engine	27
4.4.6. Narzędzia badawcze	28
5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym	29
5.1. Architektura i metodologia systemu benchmarkowego	29
5.1.1. Komponenty systemu testowego	29
5.1.2. Zbiór danych testowych	30
5.1.3. Metryki oceny i wzór composite score	31
5.1.4. Dostrajanie parametrów pracy modelu	32
5.1.5. Modele kandydujące	32
5.2. Etap Optuna: chłodzenie, hiperparametry i dominacja halucynacji	33
5.2.1. Warunek termiczny pomiaru	33
5.2.2. Dominacja flagi halucynacji	34
5.2.3. Opóźnienie E2E i model Gaussa	37

5.3. Etap kwantyzacji: cold vs warm i model log-normalny	39
5.3.1. Typy kwantyzacji GGUF	39
5.3.2. Wstępne pomiary E2E i mechanizm cold vs warm cache	39
5.3.3. Model log-normalny dla czasów generacji	42
5.3.4. Wyniki warm-cache	44
5.3.5. Anomalie i wnioski	47
5.4. Prompt engineering i prefix caching	47
5.4.1. Prefix caching promptu systemowego	47
5.4.2. Budowa promptu systemowego	48
5.5. Analiza wyników i rekomendacja produkcyjna	49
5.5.1. Podsumowanie badań	49
5.5.2. Rekomendacja produkcyjna	50
6. Realizacja prototypu	52
6.1. Architektura prototypu	52
6.2. Elementy systemu i wykorzystane technologie	53
6.2.1. Aplikacja użytkownika	53
6.2.2. Transkrypcja mowy (STT)	56
6.2.3. Context Engine	56
6.2.4. LLM Nauczyciel	57
6.2.5. SLM Uczeń	58
6.2.6. Synteza mowy (TTS)	59
6.2.7. Konfiguracja	59
6.3. Testy rozwiązania	59
7. Podsumowanie	60
Bibliografia	61
Wykaz symboli i skrótów	64
Spis rysunków	65
Spis tabel	66
Spis załączników	66

1. Wstęp

1.1. Tło problemu i cel pracy

Praca dotyczy zaprojektowania i implementacji asystenta głosowego. Rozwiązanie to ma służyć osobom przebywającym w domkach wczasowych na terenie ośrodka wypoczynkowego Politechniki Warszawskiej w Wildze. Uczelnia prowadzi w domkach wczasowych badania, wprowadzając rozwiązania IoT, co prowadzi do powstania inteligentnej przestrzeni edukacyjnej typu *living lab*. Głównym celem prac prowadzonych w ramach tej pracy magisterskiej będzie wdrożenie rozwiązania, które pozwoli wczasowiczom zdobyć szersze zrozumienie danych zbieranych w domku wczasowym. Interakcja z asystentem powinna być naturalna i płynna oraz intuicyjna dzięki jego głosowemu interfejsowi. Asystent zostanie oparty na otwartoźródłowych modelach LLM, a praca skupi się na porównaniu dostępnych technologii oraz ich analizie w kontekście wydajności pracy na sprzęcie z ograniczonymi zasobami obliczeniowymi.

Środowiskiem docelowym dla realizowanego projektu jest Ośrodek Wypoczynkowy Politechniki Warszawskiej w Wildze, modernizowany w ramach projektu Wilga MiLL. Miejsce to, dzięki wprowadzaniu rozwiązań Internetu Rzeczy (IoT) do domków wczasowych, łączy funkcję rekreacyjną z zapleczem technicznym typu *living lab*. Wdrożona tam infrastruktura IoT funkcjonowała stabilnie przez długi czas, zapewniając ciągły monitoring parametrów środowiskowych oraz zużycia energii elektrycznej. Przez ten czas zespołowi badawczemu udało się zgromadzić wystarczającą ilość danych do analizy, które zostały wykorzystane w niniejszej pracy.

System czujników generuje znaczne ilości danych surowych, trudnych do analizy dla użytkownika systemu – wczasowicza. Choć są one bezcenne z perspektywy badawczej, dla przeciętnego wizytora, pozostają one niezrozumiałe i trudne do samodzielnej interpretacji. Przytłaczająca ilość informacji oraz brak intuicyjnego interfejsu mogą prowadzić do ignorowania potencjału systemu, a cel projektu, jakim jest zwiększenie świadomości o zużyciu energetycznym i wpływie człowieka na środowisko, staje się trudny do osiągnięcia bez odpowiedniego przetworzenia danych. Problem ten był już wcześniej obiektem badań zespołu badawczego. Aby zwiększyć dostępność informacji zbieranych przez system, zdecydowano się zaimplementować aplikację, która pomogłaby dostarczyć dane do użytkowników końcowych w sposób zrozumiały i intuicyjny. Niniejsza praca magisterska skupia się na rozwinięciu dostępnych interfejsów o asystenta głosowego, który podniósłby zainteresowanie systemem IoT.

Istotnym aspektem warunków pracy projektowanego rozwiązania są również zachowania użytkowników. Dotychczasowe doświadczenia zebrane przez zespół badawczy w ośrodku wskazują na sceptyczny stosunek wczasowiczów do otaczającej ich w domkach technologii. Zauważono przypadki postrzegania urządzeń IoT jako zbędne, skomplikowane lub naruszające ich prywatność, co prowadziło do odłączania ich przez wczasowiczów od zasilania. Takie akcje podejmowane przez użytkowników w sposób oczywisty dezorganizują pracę systemu IoT. Wskazuje to na konieczność podniesienia atrakcyjności rozwiązania oraz wprowadzenia poczucia bycia aktywnym użytkownikiem systemu, a nie

bycia obserwowanym przez czujniki. Interakcja z systemem nie może być dla wczasowicza obciążeniem, lecz atrakcyjnym elementem pobytu w Wildze. System powinien w sposób subtelny dostarczać ciekawych (dla osoby nietechnicznej) informacji, zachęcając do wejścia w interakcję z inteligentnym otoczeniem, zamiast prowokować do wyłączania węzłów systemu.

1.2. Teza badawcza

Z doświadczeń zebranych przez zespół badawczy wynika, że wczasowicze wykazują zainteresowanie systemem. Nie pozostają wobec niego obojętni, zwraca on ich uwagę. Problemem jest nieintuicyjność proponowanych rozwiązań. Kluczowe jest zbudowanie rozwiązania, dzięki któremu osoby będą chętnie edukowały się w zakresie ekologii oraz ich wpływu na środowisko. Dane zbierane przez system mają charakter techniczny, nie są one czytelne dla użytkownika końcowego. Zespół stale podejmuje wysiłki w celu podniesienia atrakcyjności *living lab*. W pracy przyjęto założenie, że odpowiednie przedstawienie danych zbieranych przez system IoT może podnieść zainteresowanie proponowanym rozwiązaniem.

Na tej podstawie zbudowano tezę badawczą mówiącą, że asystent głosowy posiadający dynamicznie budowany kontekst oparty o dane czujnikowe może być atrakcyjnym elementem systemu. Co za tym idzie, może pomóc osobom nietechnicznym wejść w interakcję z systemem i zagłębić się w tematykę badań prowadzonych w Wildze. Interfejs głosowy powinien być naturalny oraz intuicyjnie przekazywać informacje, jego sposób odpowiedzi musi być przystępny dla osób nietechnicznych oraz sceptyków systemu.

Drugą płaszczyzną zawartą w tezie badawczej jest wymiar sprzętowy. Poczyniono założenie mówiące, że możliwe jest zbudowanie takiego rozwiązania w oparciu o platformę o bardzo ograniczonych zasobach. Założono pracę z Raspberry Pi 5. Ten mikrokomputer często uznawany jest w świecie IoT za sprzęt względnie szybki oraz mocny. Warto jednak pamiętać, że niniejsza praca magisterska dotyczy uruchamiania modeli LLM, zadanie to wymaga bardzo dużych zasobów obliczeniowych. W takich warunkach Raspberry Pi 5 wydaje się być platformą o bardzo mocno ograniczonych zasobach. Istotnym elementem pracy jest znalezienie balansu pomiędzy jakością odpowiedzi a czasem jej generowania.

1.3. Zakres pracy

Prace skupiają się zarówno na implementacji systemu asystenta głosowego jak i na przeprowadzeniu badań dotyczących uruchamiania modeli językowych na sprzęcie o ograniczonych zasobach. Celem implementacji jest zbudowanie systemu, który pozwoli dynamicznie budować kontekst dostarczany do asystenta oraz zbudowanie interfejsu głosowego asystenta, który umożliwi użytkownikom sprawną komunikację. Celem badań jest zrozumienie istotnych aspektów uruchamiania modeli LLM na Raspberry Pi 5, znalezienie optymalnego rozwiązania z punktu widzenia balansu między czasem odpowiedzi a jej jakością.

Prace podzielono na kilka etapów. Pierwszym z nich jest przegląd literatury. Ze względu na część badawczą pracy ten zakres prac uznano za istotny. Generatywna sztuczna inteligencja, której częścią są modele LLM, to bardzo dynamicznie rozwijająca się dziedzina. Zbadanie literatury w tym temacie pozwoli przeprowadzić badania w sposób zorganizowany oraz uporządkowany.

Po dokonaniu analizy literatury przystąpiono do zbudowania koncepcji architektonicznej rozwiązania. Dzięki zbudowaniu stałych założeń węzłów systemu możliwe było jednocześnie rozwijanie systemu oraz prowadzenie badań nad jednym konkretnym węzłem- Raspberry Pi 5, na który optymalizowano prace modelu LLM.

Następny etap pracy miał charakter badawczy. Skupiono się w nim na doborze i optymalizacji konfiguracji pracy modelu językowego uruchamianego na platformie o ograniczonych zasobach. Jako przykład takiej platformy stosowano oczywiście Raspberry Pi 5. Analizie poddano między innymi wpływ parametrów uruchomieniowych modelu, kwantyzacji i wielkości modelu, sposobu budowania promptu oraz warunków pracy urządzenia na jakość generowanej odpowiedzi. Prace skupiały się na ocenie odpowiedzi w języku polskim, ponieważ z projektowanego rozwiązania będą korzystać docelowo osoby polskojęzyczne.

Po wykonaniu badań oraz opracowaniu koncepcji architektonicznych przystąpiono do implementacji systemu asystenta głosowego. System zbudowano z wykorzystaniem wniosków płynących z części badawczej oraz korzystając z założeń architektonicznych poczynionych we wcześniejszych etapach. Celem tego etapu było połączenie poszczególnych komponentów w spójny przepływ danych. Obejmował on przejście od głosowego zapytania użytkownika, przez przetworzenie wypowiedzi na tekst, zbudowanie promptu zapytania z dynamicznie budowanym kontekstem, uruchomienie lokalnego modelu językowego, aż po wygenerowanie odpowiedzi i jej odtworzenie w postaci mowy. W pracy uwzględniono również warstwę danych, ponieważ system IoT w Wildze nie był aktywny w trakcie prowadzenia prac. Zbudowano zatem warstwę symulującą przyspieszony upływ czasu, tak by kontekst zmieniał się szybciej niż w czasie rzeczywistym.

Ostatnim etapem było sformułowanie wniosków wynikających z przeprowadzonych testów. Część implementacyjna ograniczała się do budowania asystenta dla konkretnego celu- edukowania wczasowiczów w ośrodku w Wildze. Wnioski, które praca pozwoliła sformułować mają jednak charakter ogólny. Część badawcza skupiła się na problemie ogólnym, dotyczącym wielu zastosowań, co pozwoliło sformułować wnioski o charakterze ogólnym.

2. Przegląd literatury i analiza rozwiązań

2.1. Architektury hybrydowe Edge–Cloud

Asystent głosowy oparty na modelu LLM może działać na dwa sposoby: lokalnie na urządzeniu brzegowym (ang. *edge computing*) lub po stronie serwera, w chmurze (ang. *cloud computing*). Obie z metod kryją za sobą wady oraz zalety, które mają bezpośredni wpływ na sposób funkcjonowania systemu AI.

Architektura chmurowa może oferować bardzo duże zasoby obliczeniowe, zwykle zdecydowanie większe niż lokalnie działające urządzenia brzegowe czy serwery prywatne. Niewątpliwą wadą tych rozwiązań jest niezerowe opóźnienie komunikacyjne wynikające z konieczności transmitowania danych przez sieć internetową. Drugim uwarunkowaniem znaczącym w kontekście domków w Wildzie jest konieczność stałego podłączenia do internetu, aby przysyłać dane do chmury. Wadą, o której warto pamiętać są kwestie prywatności, szczególnie przy ograniczaniu kosztów wielu providerów mocy obliczeniowej może okazać się nieuczciwa. W Wildzie użytkownicy są obserwowani przez system w trybie ciągłym, wysyłanie danych z tego typu monitoringu może budzić uzasadnione obawy w kwestiach prywatności [1].

Architektura brzegowa umożliwia przetwarzanie lokalne, oczywiście eliminuje to problemy z latencją oraz bezpieczeństwem i prywatnością danych. Ciągnie za sobą jednak inne ograniczenia. Często urządzenia brzegowe dysponują mniejszymi możliwościami obliczeniowymi co powoduje, że używane modele muszą być mniejszych rozmiarów. Badania empiryczne pokazują, że procesory ARM CPU (taki jak Raspberry Pi 5) osiągają wydajność rzędu od 5 do 15 tokenów generowanych na sekundę dla modeli o wielkości w granicach 1,5 miliarda parametrów oraz od 2 do 15 tokenów na sekundę dla modeli posiadających 3 miliardy parametrów [2]. Warto mieć na uwadze, że wąskim gardłem, dominującym ograniczeniem pozostaje przepustowość pamięci (ang. *memory bandwidth*) oraz konieczność utrzymania temperatury procesora pod kontrolą [3].

Istnieje kompromis- architektura hybrydowa, w której trudniejsze rozumowanie (ang. *reasoning*) realizowane jest na lokalnym serwerze, natomiast inferencja (proces generowania odpowiedzi) obsługująca użytkownika bezpośrednio odbywa się na urządzeniu brzegowym. Podejście to zachowuje prywatność danych (serwer działa lokalnie), jednocześnie zapewniając dostęp do modeli wyższej jakości na potrzeby złożonej analizy [1]. Oczywiście jest to kompromis, modele uruchamiane na lokalnym serwerze nie są tak rozbudowane jak te uruchamiane w chmurze. Na takim modelu postanowiono się oprzeć podczas projektowania architektury rozwiązania, co opisano w rozdziale 3.2.

Fundamentem współczesnego przetwarzania języka naturalnego (ang. *Natural Language Processing*, NLP) pozostaje architektura Transformer [4]. To właśnie na niej opierają się niemal wszystkie stosowane obecnie modele językowe popularnych dostawców. W ostatnich latach widoczny zaczyna być trend rozwijania małych modeli językowych (ang. *Small Language Models*, SLM), liczącym poniżej 8 miliardów parametrów. Dzięki treningowi prowadzonemu na wysokiej jakości, selektywnie dobranych danych modele te niekiedy

dorównują jednostkom wielokrotnie od nich większym. Potwierdzają to raporty techniczne opublikowane przez firmy Meta [5], Microsoft [6] oraz Google [7]: modele Llama 3, Phi-3 i Gemma 2 osiągają wyniki konkurencyjne względem modeli o rozmiarach o wiele większych, rzędu dziesiątek miliardów parametrów, przy ułamku ich wymagań pamięciowych. Wymienione modele dostarczane są przez wyróżniających się od lat graczy rynku IT: Meta, Microsoft i Google. Wielkość i renoma wymienionych firm pokazuje popularyzację modeli SLM. Uzupełnieniem tego nurtu jest Bielik [8], polski model oparty na architekturze Mistral, rozwijany w ramach konsorcjum SpeakLeash. Rynek małych modeli językowych rozwija się bardzo dynamicznie. Nawet w trakcie realizacji niniejszej pracy pojawiły się kolejne generacje modeli znanych dostawców (Gemma 4, Google), co wymuszało bieżące korygowanie decyzji dotyczących zakresu ewaluacji.

Trudność związaną z dynamiką rynku opisali Renney i in. [1]. Ich badania wskazują na wysoką wymiarowość przestrzeni konfiguracji jako istotne utrudnienie w badaniach nad edge-computing. Badacze, wskazują że konfiguracja optymalna dla jednej generacji modeli może nie być transferowalna na kolejną generację, nawet tego samego providera. Z tego względu w niniejszej pracy zdecydowano się wprowadzić metodologię benchmarku, oparta na automatycznej ewaluacji typu LLM-as-judge oraz kompozytowym wskaźniku jakości, została zaprojektowana jako niezależna od konkretnego modelu (ang. *model-agnostic*) i powtarzalna bez konieczności modyfikacji infrastruktury.

Platforma Raspberry Pi 5 jest wyposażona w 8 GB pamięci RAM oraz procesor ARM Cortex-A76. Modele o wielkości od 2 do 4 miliardów parametrów stanowi popularne optimum. Modele mniejsze niż 1,5 miliarda parametrów często nie osiągają zadowalającej jakości odpowiedzi w języku polskim, natomiast modele przekraczające 4 miliardy parametrów wyczerpują możliwości obliczeniowe jednostki CPU lub generują odpowiedź w nieakceptowalnie długim czasie w kontekście rozmowy głosowej [2].

2.2. Kwantyzacja dla inferencji na sprzęcie bez GPU

Kwantyzacja jest popularną techniką redukcji precyzji wag modeli LLM w celu zmniejszenia jego wielkości i zmniejszenia opóźnień odpowiedzi modelu. Znajduje ona swoje zastosowanie w dwóch odrębnych kontekstach: procesie trenowania (fine tuning) oraz optymalizacji samej inferencji (przyspieszenie prędkości generowania). Niniejsza podsekcja dotyczy drugiego zastosowania, zmniejszenia precyzji wag gotowego modelu w celu przyspieszenia generacji tokenów i redukcji zużycia zasobów platformy sprzętowej podczas inferencji.

Ograniczeniem występującym w procesie inferencji dużych modeli językowych jest przepustowość pamięci (ang. *memory-bandwidth-bound*), a nie wydajnością obliczeniową. Kim et al. [9] wykazali, że generowanie każdego tokenu wymaga jednorazowego załadowania wszystkich wag modelu z pamięci RAM do rejestrów procesora, wobec czego w przybliżeniu opóźnienie inferencji maleje w liniowo wraz ze zmniejszaniem precyzji bitowej wag (kwantyzacja). W środowisku CPU o architekturze ARM pozbawionym GPU, gdzie przepustowość pamięci wynosi ok. 50 GB/s (tak jak w Raspberry Pi 5), model o wielkości 4 miliardów parametrów w precyzji FP16 (16 bit) zajmuje ok. 8 GB i musi być odczytany przy

generowaniu każdego tokenu. Po kwantyzacji do 4-bitów na wagę (Q4_K_M) zajmuje on ok. 2,5 GB, co przekłada się na około trzykrotnie wyższą prędkość generacji (tokenów na sekundę).

Frantar et al. [10] zaproponowali metodę GPTQ (ang. *Generative Pre-Trained Transformers Quantization*). Metoda ta polega na przeprowadzeniu procesu kwantyzacji jednokrotnie dopiero po zakończeniu trenowania (ang. *post-training quantization*, PTQ). Metoda ta pozwala skwantyzować model o wielkości 175 miliardów parametrów do 3 lub 4 bitów w około cztery godziny pracy GPU, efektem końcowym jest model znacznie lżejszy o pomijalnym spadku jakości generowanej odpowiedzi. GPTQ to fundament formatu GGUF stosowanego szeroko w świecie LLM i SLM. Format ten jest wykorzystywany między innymi przez niezwykle popularną bibliotekę llama.cpp. Oprogramowanie to jest domyślnym silnikiem inferencji w aplikacji Ollama, które zostało wykorzystane w niniejszej pracy magisterskiej.

Format GGUF (ang. *GPT-Generated Unified Format*) przechowuje wagi w, tak zwanych, zgrupowanych schematach kwantyzacji (ang. *groupwise quantization*), w których grupy wag potrafią współdzielić współczynnik skalowania. Poszczególne schematy różnią się precyzją: Q4_K_M (4 bity) może niejednokrotnie stanowić kompromis między rozmiarem a jakością; Q8_0 (8 bitów) zapewnia wyższą precyzję kosztem większego zużycia pamięci; natomiast Q2_K i Q3_K agresywnie redukują rozmiar przy zauważalnym spadku jakości [11]. Schemat Q4_K_M stał się bardzo popularnym standardem wdrożeń na sprzęcie o ograniczonych zasobach, w tym na architekturze ARM [1], którą charakteryzuje się Raspberry Pi. W modelach MoE (ang. *Mixture-of-Experts*) agresywna kwantyzacja do Q2_K może natomiast powodować znaczący spadek jakości. Dzieje się tak ze względu na wrażliwość mechanizmu komunikacji między ekspertami, błąd jest propagowany przez co rośnie.

Warto zaznaczyć, że kwantyzacja jest skuteczną metodą zmniejszania czasu generacji na urządzeniach o ograniczonych zasobach. Jest tak, ponieważ głównym ograniczeniem pozostaje przepustowość pamięci, redukcja precyzji wag zmniejsza wolumen danych odczytywanych przy każdym tokenie. W praktyce powoduje to skrócenie latencji inferencji. Na platformach brzegowych dodatkowym ograniczeniem pozostaje też utrzymanie temperatury procesora pod kontrolą [3].

2.3. Wielomodelowe systemy LLM - podział ról

Hierarchiczny podział ról między modelami jest pojęciem wywodzącym się z destylacji wiedzy (ang. *knowledge distillation*). Zostało ono wprowadzone przez Hintona et al. [12]. Opiera się ono na przekazywaniu wiedzy między modelami. Duży model nauczyciel (ang. *teacher*) transferuje wiedzę do mniejszego modelu- ucznia (ang. *student*) w procesie trenowania. Dzieje się to poprzez udostępnianie miękkich etykiet (ang. *soft labels*) zamiast konkretnych (twardych) odpowiedzi. Podejście to wykorzystuje wyraźną hierarchię zdolności. Nauczyciel (większy model) dysponuje bogatszą wiedzą, a uczeń (model mniejszy) stara się odwzorować jego zachowanie przy istotnie mniejszych wymaganiach sprzętowych. Autorzy wykazali, że tak przeprowadzony proces transferu wiedzy pozwala stosunkowo ma-

łemu modelowi osiągnąć jakość zbliżoną do modelu źródłowego, co uzasadnia zasadność koncepcji współpracy modeli o różnej wielkości.

Koncepcja dynamicznego przydziału zadań rozwinęła się dalej. Jej rozwinięciem jest paradygmat kaskadowego wnioskowania (ang. *LLM cascading*). Chen et al. [13] pokazali, że sekwencyjne odpytywanie modeli w odpowiedniej kolejności, wraz z zatrzymaniem po osiągnięciu progu jakości odpowiedzi – redukuje czas inferencji nawet o 98% przy jakości porównywalnej z korzystaniem wyłącznie z największego modelu. W tym podejściu podział ról nie jest ustalony z góry (nauczyciel uczeń), lecz wynika z trudności konkretnego zapytania. Proste przypadki obsługuje tańszy, mniejszy model, a jedynie zadania trudniejsze trafiają do jednostki o większych możliwościach.

Jeszcze innym wzorcem współpracy jest dekodowanie spekulatywne (ang. *speculative decoding*), zaproponowane przez Leviathana et al. [14]. Mały i szybki model tzw. model szkicowy (ang. *draft model*) proponuje ciąg kolejnych tokenów (układ słów w odpowiedzi), a duży model docelowy weryfikuje je w czasie generacji w jednym kroku, akceptując albo odrzucając kolejne propozycje. Autorzy wykazali, że taka procedura przyspiesza generację, ponieważ większy model jest jedynie recenzentem już gotowej odpowiedzi. Podział ról wynika tu z charakterystyki modeli, a nie z trudności zadania. Mały model odpowiada szybciej, więc jest użyty do generowania szkicu, duży zaś odpowiada za zachowanie jakości.

2.4. Inżynieria promptów i dostarczanie kontekstu

Inżynieria promptów (ang. *prompt engineering*) to pojęcie kryjące pod sobą naukę o sposobie kształtowania zapytań trafiających do modelu językowego. Ma ona na celu możliwą poprawę jakości odpowiedzi oraz wymuszeniu ich formy [15]. Schulhoff et al. dokonali analizy taksonomii pięćdziesięciu technik promptowania, wyróżniają oni między innymi, które określają jako *zero-shot* charakteryzuje się ono brakiem przykładów przekazanych w prompcie. Podejście z kilkoma przykładami umieszczonymi w prompcie- *few-shot* oraz z krokami rozumowania *chain-of-thought*. Zbiór tych technik pokazuje, że sposób sformułowania zapytania stanowi istotny wymiar sterowania odpowiedzią modelem, niezależny od jego wag. Brown et al. [16] pokazali, że duże modele adaptują zachowanie do zadania tylko i wyłącznie na podstawie przykładów zawartych w prompcie. Możliwe jest więc pseudo uczenie bez modyfikacji wag. Zjawisko to określa się jako uczenie w kontekście (ang. *in-context learning*). Wei et al. [17] pokazali natomiast, że wymuszenie kolejnych kroków rozumowania (*chain-of-thought*) znacząco poprawia jakość na zadaniach wymagających wieloetapowości. Nawet bez fine-tuningu modelu. Obserwacje te dowodzą, że sama konstrukcja wiadomości wejściowej może w znaczącym stopniu zastąpić proces treningu. Modyfikację parametrów może być czaso i zasobochłonna, ma szczególne znaczenie w scenariuszach o ograniczonych zasobach, z natury kojarzącymi się z ograniczonym budżetem.

W kontekście SLM, małych modeli językowe (ang. *small language model*) działających na urządzeniach brzegowych szczególnego znaczenia nabiera kontrolowanie odpowiedzi oraz zachowania modelu (ang. *constraining*). Najczęściej wykonuje się to za pomocą system promptu, czyli stałej części wiadomości wejściowej. Modele o rozmiarze rzędu

od 2 do 4 miliardów parametrów wykazują skłonność do generowania informacji nieobecnych w dostarczonym kontekście, czyli do halucynacji (ang. *hallucination*). Przy braku dodatkowych mechanizmów zabezpieczających może podważać to wiarygodność asystenta. Odpowiednio sformułowany system prompt może wymuszać określoną politykę zachowania na przykład transparentne komunikowanie braku danych, gdy model nie ma danych, o które jest pytany. Skutecznie prowadzi to do ograniczenia halucynacji bez konieczności prowadzenia procesu treningowego (zmienianie wag modelu).

Niezwykle powszechnie stosowanym sposobem dostarczania wiedzy do modelu jest podejście Retrieval-Augmented Generation (RAG). Podejście to zostało zdefiniowane przez Lewisa et al. [18] jako rozwiązanie łączące wiedzy zapisaną w wagach modelu z wiedzą dostarczoną zewnętrznym poprzez dobieranie dynamicznego kontekstu z danych (ang. *retrieval*). Klasyczny RAG pobiera pliki z bazy wektorowej na podstawie podobieństwa semantycznego aby następnie dołączyć je do kontekstu zapytania. W opisywanym w tej pracy systemie dane wstrzykiwane są do promptu deterministycznie. Nie jest przy tym wykonywana operacja wyszukiwania – kontekst jest każdorazowo konstruowany od podstaw z surowych danych. Takie podejście można określić mianem strukturyzowanego wstrzykiwania kontekstu (ang. *structured context injection*), które to czerpie z metodologii RAG. Podejście prezentowane przez [18] pokazuje wpływ odpowiedniego doboru kontekstu na jakość odpowiedzi.

2.5. Optymalizacja hiperparametrów generacji

Parametry pracy modelu językowego takie jak: temperatura próbkowania (ang. *sampling temperature*), rozmiar okna kontekstu (ang. *context window*), maksymalna długość odpowiedzi, współczynnik top-p oraz kara za powtórzenia mają wpływ na jakość generowanej odpowiedzi oraz czas jej generacji. Optymalne wartości parametrów nie są jednak stałe dla modelu, zależą m.in. od zadania i sprzętu. Przeszukiwanie pełnej siatki kombinacji wymienionych parametrów jest kosztowne obliczeniowo, nawet niewielka przestrzeń parametrów może dać bardzo wiele kombinacji, co powoduje, że testowanie ich wszystkich jest czasowo nieopłacalne.

Autorzy pracy [19] przedstawiają bibliotekę Optuna: framework optymalizacji hiperparametrów oparty na bayesowskim próbkowaniu z algorytmem TPE (ang. *Tree-structured Parzen Estimator*). Zamiast testować kombinacje losowo lub w zadanej kolejności, algorytm TPE buduje probabilistyczny model funkcji celu na podstawie zebranych dotychczas wyników i kieruje kolejne próby w kierunku wykazującym największe obietnice nagrody. Metoda ta osiąga porównywalną eksplorację przy znacznie mniejszej liczbie ewaluacji. Metoda ta jest skuteczna w wielu rodzajach optymalizacji siatki wieloparametrowej. Znajduje swoje zastosowanie również w szukaniu optymalnej konfiguracji parametrów pracy modelu SLM z wykorzystaniem aplikacji Ollama.

2.6. Fine tuning modeli językowych

Kwantyzacja omówiona w sekcji powyżej dotyczy redukcji precyzji wag gotowego modelu wyłącznie w celu optymalizacji procesu generowania, *fine-tuning* (dostrajanie) polega na modyfikacji wartości wag. Celem jest adaptacja modelu do konkretnych zadań poprzez aktualizację parametrów w procesie treningu.

Pełne dostrajanie (*full fine-tuning*) modeli posiadających miliardy parametrów jest niepraktyczne, szczególnie jeśli jest przeprowadzane lokalnie. Autorzy pracy [20] zaproponowali metodę LoRA (ang. *Low-Rank Adaptation*), nie skupia się ona na modyfikacji wszystkich wag, zamiast tego polega na dodaniu par małych macierzy o niskim znaczeniu obliczeniowym (ang. *low-rank matrices*), których iloczyn wyznacza zmianę wag, podczas gdy wagi bazowe pozostają zamrożone. LoRA redukuje liczbę trenowalnych parametrów nawet 10 000-krotnie (dla modelu GPT-3 zmniejsza to liczbę parametrów poddanych treningowi o 175 miliardów parametrów), przy zachowaniu jakości porównywalnej lub nawet wyższej od pełnego treningu na standardowych zadaniach NLP. Dodatkowe macierze można scalić z wagami bazowymi po zakończonym treningu, eliminując niepotrzebny narzut podczas inferencji.

Dettmers et al. [21] rozszerzyli metodę LoRA o kwantyzację wag modelu bazowego podczas trenowania. Nowo stworzoną metodę nazwali QLoRA, w niej wagi zamrożonego modelu bazowego przechowywane są w precyzji 4-bitowej, natomiast gradienty obliczane są w wyższej precyzji dla zachowania stabilności numerycznej. Umożliwia to trenowanie (*fine-tune*) modeli o 7 miliardów parametrów na GPU z 24GB pamięci VRAM, a nawet modeli 65 miliardów parametrów na 48GB VRAM. W kontekście niniejszej pracy QLoRA wydaje się być niezwykle atrakcyjnym sposobem prowadzenia douczania modelu SLM.

2.7. Interfejsy głosowe: rozpoznawanie mowy, detekcja aktywności głosowej i synteza mowy

Przepływ danych w systemie głosowym wymaga mechanizmu detekcji aktywności głosowej (ang. *Voice Activity Detection*, VAD), odróżniającego segmenty mowy od ciszy i szumów tła. Bez VAD model odpowiedzialny za rozpoznawanie mowy przetwarzałby strumień audio w sposób ciągły. Takie rozwiązanie byłoby nieakceptowalne ponieważ może generować fałszywe transkrypcje z segmentów ciszy i zwiększając latency. Zastosowanie jasnego sygnału startowego (ang. *start-of-speech*) pozwala na ograniczenie niepotrzebnego przetwarzania. Silero Team [22] opracowało lekki model neuronowy VAD do wdrożeń brzegowych (ang. *edge-computing*), przetwarza on fragmenty audio o długości około 30 ms w czasie poniżej 1 ms na jednostce CPU, przy rozmiarze modelu poniżej 1 MB. Model wspiera próbkowanie 8 kHz i 16 kHz, dostępny jest na licencji MIT oraz zoptymalizowany m.in. dla PyTorch. Takie parametry czynią z niego idealne rozwiązanie dla zastosowań IoT.

W rozpoznawaniu mowy (ang. *Speech-to-Text*, STT) modelem open-source będącym standardem rynkowym stał się Whisper, opracowany przez autorów [23]. Ten model transkrypcyjny model został wytrenowany metodą słabego nadzoru (ang. *weak supervision*) na 680 000 godzin wielojęzycznego audio. Dzięki takiej technice uzyskano wysoką odporność

na szumy tła, która jest kluczowa w warunkach domu wczasowego na terenie ośrodka w Wildze gdzie niemożliwe jest regulowanie akustyczne tła. Whisper wspiera transkrypcję języka polskiego oraz jest dostępny w kilku wariantach rozmiaru za darmo. Wariant *faster-whisper* zaprojektowany jest pod pracę na urządzeniach o ograniczonych zasobach, wydaje się on być skrojony idealnie pod opisywane rozwiązanie.

Modeli zajmujących się syntezą mowy (ang. *Text-to-Speech*, TTS) jest dziś na rynku bardzo wiele. Obok komercyjnych platform o wysokiej jakości syntezy, takich jak ElevenLabs [24], funkcjonuje wiele rozwiązań otwartoźródłowych. Klasycznym modelem end-to-end jest m.in. VITS [25], warunkowy autoenkoder wariacyjny z treningiem adversarialnym, oferujący naturalne brzmienie przy niskim opóźnieniu. Innym przykładem jest biblioteka `edge-tts` [26], udostępnia ona głosy znane z przeglądarki Microsoft Edge, w tym między innymi polski głos `pl-PL-ZofiaNeural`.

3. Koncepcja i architektura systemu

3.1. Założenia projektowe

Założenia projektowe oparte zostały o przygotowanie rozwiązania, które będzie użyteczne dla użytkowników przebywającego w domku wypoczynkowym. Wczasowicze to często osoby nietechniczne. Asystent głosowy nie powinien pełnić wyłącznie prezentować technologii, lecz stanowić intuicyjną warstwę dostępu do danych zbieranych przez system IoT oraz informacji w nich zawartych. Właśnie dlatego za istotne uznano nie tylko poprawne funkcjonowanie techniczne rozwiązania, ale również ogólną jakość interakcji, która rozumiana jest jako płynność rozmowy, jasność komunikatów generowanych, zgodność odpowiedzi z faktami oraz postanowiono zadbać o odpowiedni ton głosu asystenta głosowego.

W trakcie prac przyjęto założenie mówiące, że satysfakcja użytkownika końcowego zależy od wielu czynników. Krótki czas odpowiedzi pozostaje ważnym choć nie jedynym wymaganiem, ponieważ zbyt duża latencja burzy naturalność rozmowy głosowej. Również istotnym jest poprawność językowa, ograniczenie halucynacji odpowiedzi modelu, zgodność odpowiedzi z aktualnymi danymi pomiarowymi oraz przedstawiania informacji w sposób zrozumiały dla osoby, która nie zna struktury systemu technicznego. Z tego powodu projekt zakłada wykorzystanie dynamicznie budowanego kontekstu, w oparciu o głębszą analizę, która pozwala asystentowi mieć szersze rozumienie sytuacji w domku wczasowym.

Założone pracę z urządzeniem brzegowym Raspberry Pi 5. Sprzęt ten posiada ograniczone zasoby obliczeniowe. Generuje to dodatkowe wymagania, konieczne więc okazały się założenia, które system musi spełnić. Prowadzenie inferencji modelu na sprzęcie o tak ograniczonych zasobach wymaga ostrożnego doboru modeli, parametrów generacji oraz długości przekazywanego kontekstu (prompt). Dlatego konieczne jest przeprowadzenie części badawczej pracy. Uznano, że konieczne jest aby objęła ona sprawdzenie, w jakim można modyfikować mały model językowy, oraz parametry jego pracy, aby zapewniał możliwie wysoką jakość odpowiedzi przy ograniczonej mocy obliczeniowej i dostępnej pamięci.

Sednem pracy jest część badawcza oraz budowa systemu. Mając to na względzie postanowiono ograniczyć zakres prac niezwiązanych bezpośrednio z główną opisaną wyżej problematyką. Dla modułów przetwarzania mowy, takich jak detekcja aktywności głosowej (VAD), rozpoznawanie mowy (STT) oraz synteza mowy (TTS), założono wykorzystanie gotowych i zoptymalizowanych rozwiązań typu open-source. To podejście pozwoliło skoncentrować się na integracji danych z systemu IoT, mechanizmie budowania promptu, architekturze systemu oraz analizie parametrów pracy modelu SLM wpływających na skuteczność lokalnego asystenta głosowego.

3.2. Architektura Nauczyciel–Uczeń (Teacher–Student)

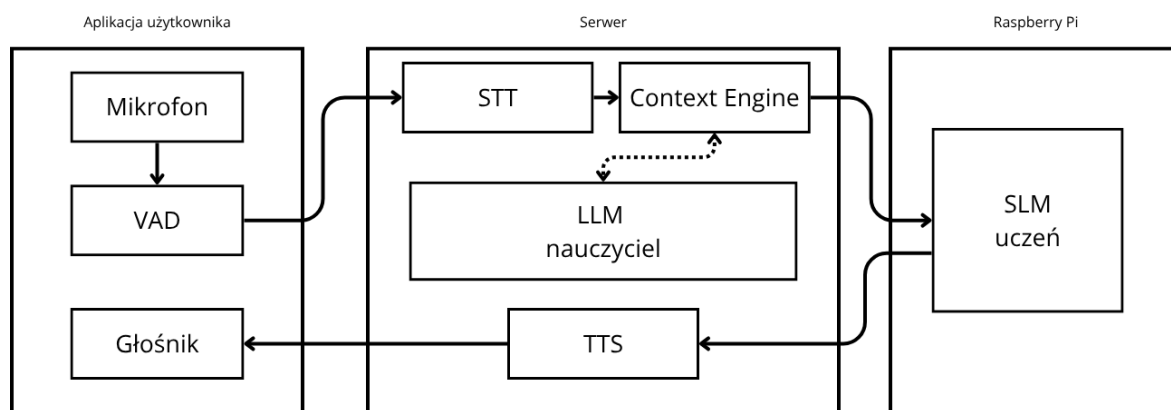
Zaprojektowano rozwiązanie oparte na architekturze hybrydowej, w której odpowiedzi generowane przez model SLM (uczeń) odbywa się lokalnie na urządzeniu brzegowym (Edge

Computing), natomiast reszta pracy wymagająca mocy obliczeniowej została przeniesiona na stronę serwera lub aplikacji użytkownika. Takie rozwiązanie powoduje maksymalne obciążenie węzła o ograniczonych zasobach w celu maksymalizacji jego performance'u. Przetwarzanie dźwięku oraz zaawansowana analiza danych historycznych realizowane są zatem użyciem infrastruktury serwerowej.

Proponowana w pracy architektura nie jest typową architekturą Nauczyciel-Uczeń (Teacher-Student) opisaną w literaturze. Nawiązuje ona do koncepcji destylacji wiedzy w systemie dwumodelowym, opisaną w pracy [12]. Nie implementuje go jednak w klasycznym ujęciu. Standardowo oznacza to trening mniejszej sieci (SLM) na podstawie wyjść sieci większej, w prezentowanej pracy magisterskiej koncepcja ta realizowana jest poprzez sekwencyjne, hierarchiczne przetwarzanie informacji, w którym model nie poddany kwantyzacji wag ("Nauczyciel") przetwarza surowe dane. Na tej podstawie generuje nieoczywiste wnioski, które przekazywane są razem z surowymi danymi do ucznia. Faza analizy danych wykonywana jest na sprzęcie o większych zasobach oraz wykonywana jest asynchronicznie, dzięki czemu nie interferuje z pracą ucznia.

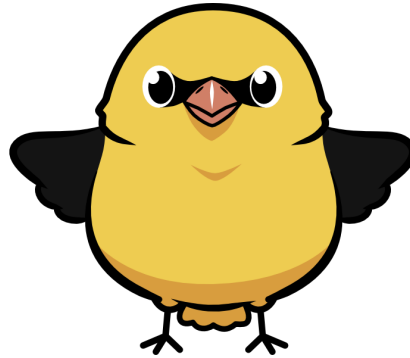
Serwer pełni rolę "Nauczyciela". Wykorzystując swoją moc obliczeniową uruchamia okresowo model językowi (o wielkości większej niż model ucznia). Nauczyciel analizuje surowe dane z całego dnia i generuje zwarte, wysokiej jakości podsumowanie w języku naturalnym. Wygenerowane przez model podsumowanie jest przesyłane razem z surowymi danymi do modelu ucznia. Dzięki takiemu rozwiązaniu mały model ucznia dostaje informacje przetworzone, oraz surowe. Te pierwsze pomagają mu dostarczać informacji ciekawych poddanych analizie, których przez swoje ograniczone modele nie umiałby wyciągnąć samodzielnie. Te drugie natomiast dają mu pełen ogłód sytuacji i pozwalają odpowiadać na konkretne pytania użytkowników końcowych. Takie rozwiązanie pozwala systemowi brzmieć na mądrzejszy przy jednoczesnym zachowaniu niskiej latencji (analiza danych wykonywana przez duży model wymaga czasu).

3.3. Schemat przepływu danych w systemie



Rysunek 3.1. Architektura systemu.

Przedstawiona powyżej architektura systemu zakłada zbudowanie aplikacji użytkownika w oparciu o rozwijaną wcześniej w ramach systemu Wilga MiLL aplikację dashboardową do prezentacji danych. W aplikacji użytkownikom zaprezentowany został Wilguś, postać oparta o ptaka- Wilgę co w oczywisty sposób nawiązuje do nazwy miejscowości, w której mieści się ośrodek wypoczynkowy PW. Wygląd postaci przedstawiono na rys. 3.2.



Rysunek 3.2. Wilguś w aplikacji użytkownika.

Rozwiązanie budowane w niniejszej pracy dziedziczy styl oraz język wizualny z wymienionej aplikacji. Docelowa integracja obu rozwiązań pozwoliłaby połączyć dwa interfejsy wizualizacji danych, wizualny z głosowym. Poczynione założenie niebudowania aplikacji od podstaw ani integracji obu rozwiązań ma podłoże praktyczne. Aplikacja wizualna zebrała bardzo przychylne oceny zespołu Wilga MiLL, jej język wizualny jest przystępny, a postać Wilgusia skutecznie budzi zaufanie oraz pozbawia użytkowników przytłaczającego wrażenia skomplikowania danych IoT. Niniejsza praca skupia się na asystencji głosowej, integracja rozwiązań wymagałaby pracy poświęconej skomplikowanemu budowaniu aplikacji webowej, zdecydowano zatem skupić się na aspektach technicznych oraz badawczych związanych z interfejsem głosowym.

W przyjętej architekturze rozwiązania aplikacja webowa nie realizuje samodzielnie rozpoznawania mowy ani też syntezy odpowiedzi. To leży odpowiednio po stronie serwera oraz węzła o ograniczonych zasobach (RPi). Zadaniem aplikacji jest obsługa interakcji użytkownika oraz przekazanie sygnału audio do warstwy serwerowej. Po stronie serwera uruchamiany jest proces odpowiedzialny za detekcję aktywności głosowej, transkrypcję mowy na tekst oraz wygenerowanie odpowiedzi głosowej na podstawie tekstu generowanego przez SLM. Rozdzielenie tych zadań uznano za rozsądne, ponieważ pozwala ono wykorzystać urządzenie końcowe o małych zasobach, oraz skorzystać przyspieszyć proces korzystając mocy obliczeniowej dostępnych na serwerze.

Centralnym elementem przeprowadzonych badań pozostał mały model językowy SLM (Small Language Model), uruchomiony na mikrokomputerze Raspberry Pi 5. Zdecydowano się badaniom poddać ten właśnie komponent, ponieważ model językowy stanowi najbardziej wymagającą część całego łańcucha przetwarzania. Wyzwanie jest tym większe, że Raspberry Pi 5 jest wyposażone jedynie w 8 GB pamięci RAM i nie posiada dedykowanej karty graficznej, co narzuca istotne ograniczenia, uruchamianie modeli typu LLM na tak ograniczonym sprzęcie jest bardzo wymagające. Praca koncentruje się na sprawdzeniu,

czy lokalnie działający model może generować odpowiedzi o jakości wystarczającej dla użytkownika końcowego, doborze modelu, jego modyfikacji oraz zbadaniu jakie warunki pracy dadzą najlepsze wyniki.

W systemie istnieją dwa modele językowe. Drugim z nich jest większy model językowy (powyżej 8 miliardów parametrów), pełniący rolę nauczyciela. Jest on uruchamiany na serwerze i odpowiada za przygotowywanie informacji pomocniczych dla modelu ucznia. Działa on asynchronicznie i nie obsługuje bezpośrednio każdej wypowiedzi użytkownika, pracuje cyklicznie przygotowując kontekst, który przekazywany jest do małego modelu. Dzięki temu model uruchomiony na RPi posiada ciekawe z punktu widzenia użytkownika końcowego informacje.

Za dostarczenie danych oraz orkiestrację procesu przepływu danych w systemie odpowiada moduł nazwany Context Engine. Pod tą nazwą w projekcie ukryto zestaw skryptów przygotowanych w języku Python, których zadaniem jest cykliczne pobieranie danych z bazy danych, kierowanie zapytań do modelu nauczyciela oraz budowanie treści wstrzykiwanej następnie do promptu modelu ucznia. Konieczność wprowadzenia takiego modułu wynika ze specyfiki działania modeli językowych, które to nie działają samoczynnie, lecz generują odpowiedź dopiero po otrzymaniu zapytania. Konieczne było zatem wprowadzenie do systemu warstwy pośredniej, która regularnie przygotowuje kontekst poprzez wgranie zapytania z wyciągiem z bazy danych do modelu nauczyciela, a następnie udostępnia go lokalnemu modelowi (wraz z danymi surowymi) gdy w systemie pojawi się zapytanie użytkownika.

Przepływ danych rozpoczyna się od wypowiedzi użytkownika zarejestrowanej w aplikacji webowej. Nagranie jest przekazywane do serwera, tam następuje jego transkrypcji, czyli przetworzenie do postaci tekstowej. Następnie zapytanie użytkownika zostaje połączone z kontekstem przygotowanym przez Context Engine (dane surowe oraz notka przygotowana przez model nauczyciela) i przekazane do modelu ucznia uruchomionego na Raspberry Pi (uczeń). Wygenerowana przez mały model odpowiedź wraca do warstwy serwerowej, gdzie przekształcona zostaje na mowę, a następnie aplikacja odtwarza odpowiedź użytkownikowi (jednocześnie wypisując ją na ekranie). Taki podział obowiązków pozwala połączyć założenie o lokalnej inferencji modelu SLM z pełnym wykorzystaniem możliwościami obliczeniowymi infrastruktury serwerowej.

4. Środowisko projektowo-badawcze

4.1. Wybór platformy brzegowej

Praca opiera się na założeniu ograniczonych zasobów platformy brzegowej. Jako sprzęt do uruchamiania modeli językowych wybrano Raspberry Pi 5 w wersji wyposażonej w 8 GB pamięci RAM. Wybór ten wynikał z kilku powiązanych ze sobą przesłanek. Urządzenia tej klasy są powszechnie wykorzystywane w rozwiązaniach Internetu Rzeczy, tak też było w projekcie Wilga MiLL. Zespół projektowy budował system IoT właśnie w oparciu o te mikrokomputery. RPi łączy niewielkie rozmiary, jest stabilne termicznie oraz posiada dedykowaną dystrybucję systemu operacyjnego Linux o nazwie Raspbian, która jest bardzo przystępna i szybko operuje na dobranym sprzęcie.

Istotnym kryterium wyboru była również ograniczona moc obliczeniowa tej platformy. Raspberry Pi 5 nie jest wyposażone w dedykowaną kartę graficzną jest urządzeniem typu CPU only. Brak GPU przeznaczonego do przyspieszania obliczeń związanych z modelami językowymi nakładał wiele ograniczeń. W konsekwencji uruchamianie modelu LLM lub SLM na takim urządzeniu wymaga dobrania odpowiedniego modelu, jego kwantyzacji oraz odpowiedniego dobrania promptu. Ograniczenia te uznano za zgodne z celem pracy, ponieważ pozwalają zbadać możliwości działania lokalnego asystenta głosowego na sprzęcie typowym dla systemów IoT.

Zastosowanie Raspberry Pi 5 jako platformy projektowej pozwala również odtworzyć warunki, w których rozwiązanie mogłoby zostać zainstalowane w domku wypoczynkowym. Urządzenie może pracować lokalnie, w domku. Znajdując się w pobliżu źródeł danych i interfejsu użytkownika, bez konieczności przekazywania każdego żądania użytkownika do zewnętrznej usługi chmurowej. Takie podejście doskonale wpisuje się w założenia mówiące o zastosowaniu architektury brzegowej, w której część obliczeń wykonywana jest możliwie blisko miejsca prowadzenia pomiarów.

4.2. Architektura sprzętowa

W projekcie zastosowano dwa typy jednostek obliczeniowych. Pierwszą z nich jest urządzenie brzegowe, czyli Raspberry Pi 5, na którym to uruchamiany jest mniejszy model językowy, ucznia. Jego zadaniem jest obsługa właściwego zapytania użytkownika. Drugi typ jednostki stanowi platforma serwerowa, przeznaczona do pracy większego modelu językowego pełniącego rolę nauczyciela. Taki podział odpowiedzialności wynika z architektury opisanej w rozdziale 3.2.

Jako jedną z możliwych platform dla modelu nauczyciela rozważono komputer znajdującą się w laboratorium LaVa na Wydziale Elektroniki i Technik Informacyjnych Politechniki Warszawskiej. Stacja robocza wyposażony jest w kartę graficzną NVIDIA GeForce RTX 3050, co czyni go rozsądnym kandydatem do uruchamiania modeli językowych. Alternatywną platformą uruchomieniową modelu nauczyciela jest komputer MacBook z SoC Apple M4, urządzenie wyposażone jest w 16 GB pamięci RAM, która dzięki architekturze Apple jest pamięcią zunifikowaną [27]. Zaletą tej klasy sprzętu jest architektura Apple Silicon, w której

pamięć współdzielona może być wykorzystywana przez różne jednostki obliczeniowe układu. Oznacza to, że VRAM układu graficznego to pełne w granicy 16GB [28]. Taka ilość pamięci ułatwia uruchamianie lokalnych modeli językowych, szczególnie w porównaniu z klasycznymi konfiguracjami gdzie ograniczeniem bardzo często bywa pojemność pamięci karty graficznej. Zdecydowano się prace implementacyjne prowadzić na Apple MacBook M4 16GB RAM.

W przyjętej topologii Raspberry Pi pełni wyłącznie rolę lokalnego węzła uruchomieniowego dla modelu ucznia. Aplikacja użytkownika komunikuje się bezpośrednio z warstwą serwerową, która odpowiada za obsługę nagrania, transkrypcję mowy na tekst, przygotowanie kontekstu oraz syntezę odpowiedzi. Rola RPi ogranicza się jedynie do generowania odpowiedzi tekstowych. Notki kontekstowe, poddane głębszej analizie, są cyklicznie generowane przez nauczyciela działającego na serwerze. Takie rozdzielenie elementów sprzętowych pozwala maksymalnie ograniczyć obciążenie urządzenia brzegowego, przy jednoczesnym zachowaniu wszystkich wymaganych funkcji systemu.

4.3. Modele LLM w edge computing'u

Uruchamianie modeli językowych na urządzeniach brzegowych stanowi istotne wyzwanie techniczne. Szczególnie przy klasie Raspberry Pi, które to posiada ograniczoną ilość pamięci operacyjnej oraz brak dedykowanego układu GPU. W przypadku przyjętej platformy obliczeniowej dostępne jest 8 GB pamięci RAM, która musi być współdzielona przez system operacyjny, środowisko uruchomieniowe, jak i sam model wraz z kontekstem wejściowym. Oznacza to, że modele LLM, których liczba parametrów może sięgać dziesiątek lub setek miliardów, nie będą w stanie poprawnie funkcjonować na tym urządzeniu.

Z tego względu w pracy skoncentrowano się na małych modelach językowych tak zwanych SLM. Modele te mają zwykle kilka miliardów parametrów, za umowną granicę w architekturze podaje się 8 miliardów parametrów. SLM mogą być uruchamiane lokalnie po zastosowaniu odpowiedniego formatu zapisu wag modelu oraz często jego kwantyzacji. Zmniejszenie precyzji wag pozwala ograniczyć rozmiar modelu i zapotrzebowanie na pamięć, może ono jednak wpływać negatywnie na jakość generowanych odpowiedzi. Dobór modelu nie może więc opierać się wyłącznie na jego rozmiarze, lecz powinien brać pod uwagę kompromis pomiędzy jakością odpowiedzi, a czasem jej generacji przy zachowaniu stabilnej pracy jednostki brzegowej.

Szczególne znaczenie w budowie asystenta głosowego ma opóźnienie, zwane często latencją. Użytkownik oczekuje odpowiedzi w czasie zbliżonym do naturalnej rozmowy, aby zachować jej płynność. Dlatego zbyt długie oczekiwanie na pierwszy token (pierwszy moment kiedy użytkownik widzi część odpowiedzi) lub zakończenie wypowiedzi może obniżyć użyteczność całego rozwiązania. Zwiększanie rozmiaru modelu lub długości promptu (kontekstu) prowadzi zazwyczaj do wzrostu zużycia pamięci, a co za tym idzie wydłużenia czasu inferencji. W związku z tym w dalszej części pracy analizie poddano nie tylko wybór konkretnego modelu, lecz również parametry jego pracy oraz sposób przygotowania promptu.

Ograniczenia środowiska edge wpłynęły również na podjęcie decyzji o zastosowaniu architektury wzorowanej na podejściu Nauczyciel-Uczeń. Model działający na urządzeniu brzegowym (uczeń) nie powinien samodzielnie analizować pełnych zbiorów danych historycznych, ponieważ wymagałoby to przekazania długiego kontekstu i zwiększyłoby czas odpowiedzi. Zamiast tego założono, że mocniejszy model nauczyciela przygotowuje syntetyczne podsumowania oparte na danych historycznych (analiza wsteczna), które następnie są wykorzystywane przez model brzegowy. Takie podejście pozwala zmniejszyć długość promptu wstrzykiwanego do modelu ucznia. Dzięki ograniczeniu ilości informacji przetwarzanych bezpośrednio na Raspberry Pi, możliwe jest ograniczenie poziomu skomplikowania zadania przy jednoczesnym zachowaniu częściowej świadomości informacji zawartych w danych historycznych, dzięki asynchronicznie budowanemu kontekstowi.

4.4. Wykorzystane oprogramowanie i technologie

4.4.1. Aplikacja użytkownika

Aplikacja użytkownika, tak jak omówiono w rozdziale 3, nie była tworzona w ramach od podstaw. Zdecydowano się zbudować aplikację w oparciu o istniejącą aplikację systemu Wilga MiLL. W konsekwencji podstawowy stos technologiczny części frontendowej został narzucony przez istniejące rozwiązanie. Aplikacja została przygotowana z wykorzystaniem biblioteki React, języka programistycznego TypeScript, tailwind CSS oraz narzędzia Vite. Komunikacja z serwerem odbywa się z użyciem protokołu HTTP.

4.4.2. Warstwa serwerowa i komunikacyjna

Warstwa serwerowa pełni funkcję pośrednią pomiędzy aplikacją użytkownika, modułami przetwarzania mowy, modelem nauczyciela, a modelem ucznia. Do jej zaimplementowania wykorzystano język Python, gdyż jest on szeroko stosowany w rozwiązaniach związanych z uczeniem maszynowym i umożliwia sprawną integrację z bibliotekami obsługującymi modele LLM jak i przetwarzanie dźwięku. Jako technologię budowy interfejsu API wskazano FastAPI, które pozwala tworzyć lekkie i szybkie usługi HTTP oraz endpointy wykorzystywane przez aplikację użytkownika.

Zastosowanie osobnej warstwy serwerowej ułatwia rozdzielenie zakresu zadań poszczególnych komponentów. Aplikacja webowa odpowiada za interakcję z użytkownikiem, serwer natomiast koordynuje przepływ danych, uruchamia moduły przetwarzania mowy i przekazuje zapytania do odpowiednich modeli (nauczyciel i uczeń). Takie podejście ogranicza złożoność części frontendowej (o czym była mowa w założeniach) oraz pozwala testować poszczególne elementy systemu niezależnie.

4.4.3. Moduły STT i TTS

TUTAJ KONIEC

Zbudowanie interfejsu głosowego wymaga zastosowania modułów odpowiedzialnych za rozpoznawanie mowy (wejście systemu) oraz wygenerowanie odpowiedzi głosowej (wyjście systemu). W projekcie założono budowę tych modułów w oparciu o gotowe rozwiązania, ponieważ główny zakres pracy dotyczy architektury systemu, współpracy modeli

językowych oraz optymalizacji pracy małego modelu na platformie brzegowej. Samodzielna implementacja procedury STT i TTS znacząco rozszerzyłaby zakres projektu, co uznano za zbędne. Dodatkowo korzystanie z gotowych rozwiązań popularnych na rynku jest szeroko stosowanym standardem w świecie IoT.

Do transkrypcji mowy przewidziano wykorzystanie rozwiązania zgodnego z modelem Whisper, który jest rozwiązaniem proponowanym przez OpenAI. Tak samo jak większość LLM korzysta z tzw. zgodności z API OpenAI, tak samo transkryptory dostosowują sposób podania odpowiedzi do modelu od tej firmy. Whisper jest dostępny w kilku wariantach wielkości, zestawionych w tabeli 4.1 [23].

Tabela 4.1. Warianty rozmiaru modelu Whisper (OpenAI).

Wariant	Parametry	Rozmiar (ok.)
tiny	39 M	~75 MB
base	74 M	~142 MB
small	244 M	~466 MB
medium	769 M	~1,5 GB
large	1550 M	~2,9 GB

Skupiono się na implementacji typu faster-whisper używając modelu small. Biblioteka faster-whisper pozwala uruchamiać transkrypcję w sposób dostosowany do lokalnych zasobów obliczeniowych. Moduł oparty na wymienionych rozwiązaniach pozwala przekształcać nagrania użytkownika na tekst, która następnie jest przekazywany na wejście modelu językowego.

Moduł TTS pełni funkcję odwrotną i umożliwia przedstawienie odpowiedzi modelu w formie dźwiękowej. Podczas doboru rozwiązania uwzględniono kilka popularnych modeli syntezy mowy, zestawionych w tabeli 4.2. Głównymi kryteriami oceny była jakość odpowiedzi, czas jej generowania oraz możliwość lokalnej ewaluacji..

Tabela 4.2. Rozwiązania TTS rozważane w projekcie.

Rozwiązanie	Typ	Język polski	Uwagi
ElevenLabs [24]	usługa chmurowa	tak	wysoka jakość, koszt i zależność od API
VITS [25]	model lokalny	wymaga modelu	naturalne brzmienie, większy nakład wdrożenia
edge-tts [26]	głosy MS Edge	tak (ZofiaNeural)	prosta integracja, niskie opóźnienie

Na tym etapie zdecydowano się oprzeć na bibliotece edge-tts ze względu na dostępność damskiego głosu (ZofiaNeural), który uznano za bardziej odpowiedni dla postaci Wilgusi. Ponadto jakość odpowiedzi jest zadowalająca, a integracja niezwykle prosta. Ponownie, praca nie skupia się na modułach TTS i STT, stąd maksymalne uproszczenia przy zachowaniu jakości skutecznie spełniają cele pracy.

4.4.4. LLM

Do uruchamiania modeli językowych wykorzystano środowisko Ollama. Oprogramowanie to umożliwia lokalne uruchamianie modeli LLM i SLM oraz wystawianie prostego interfejsu API, pozwala to na łatwą integrację z pozostałymi komponentami systemu. Zastosowanie jednego środowiska uruchomieniowego dla modelu ucznia i modelu nauczyciela pozwoliło ograniczyć liczbę różnic konfiguracyjnych pomiędzy platformami oraz uprościć przebieg testów.

Istotną zaletą wykorzystania Ollama jest możliwość ustawiania parametrów pracy modeli językowych. Parametry te mają wpływ zarówno na jakość odpowiedzi, jak i na czas jej generacji; najważniejsze z nich, wraz z zakresami stosowanymi w części badawczej, zestawiono w tabeli 4.3.

Tabela 4.3. Parametry generacji Ollama wykorzystywane w pracy.

Parametr	Zakres	Znaczenie
temperature	[0,10; 0,80]	Steruje losowością próbkowania: niższe wartości dają bardziej deterministyczne odpowiedzi, wyższe zwiększają różnorodność.
num_ctx	{1024, 2048, 4096}	Maksymalna długość kontekstu (prompt + historia) w tokenach; zwiększenie długości kontekstu zwiększa zużycie pamięci RAM.
num_predict	[48; 160]	Limit długości generowanej odpowiedzi w tokenach; ogranicza czas inferencji i długość wypowiedzi TTS.
top_p	[0,80; 0,99]	Ogranicza wybór kolejnego tokena do najbardziej prawdopodobnych kandydatów (im niższe p , tym węższy wybór).
repeat_penalty	[1,00; 1,15]	Kara za powtarzanie już wygenerowanych tokenów; wartości > 1 ograniczają zapętlenia w odpowiedzi.

4.4.5. Context Engine

Ten moduł jest niezwykle istotny dla działania systemu. Jego zadaniem jest dynamiczne budowanie kontekstu dla małego modelu językowego- ucznia. Dzieje się to poprzez deterministyczne budowanie promptu w oparciu o dane zbierane w systemie IoT oraz porcję danych poddanych analizie przez model nauczyciela. Moduł cyklicznie pobiera dane przesyła je do modelu nauczyciela, wraz z promptem proszącym o ich analizę. Gdy pojawia się zapytanie użytkownika Context Engine buduje prompt w wyżej opisany sposób i wysyła zapytanie do ucznia. Po otrzymaniu odpowiedzi przekazuje ją do modułu TTS aby ta trafiła do użytkownika.

Context Engine został zaimplementowany jako zestaw skryptów w języku Python. Skrypty te uruchamiane są cyklicznie, dzięki czemu kontekst przekazywany do modelu ucznia jest stale aktualizowany i nie wpływa na bieżące rozmowy użytkownika. Dzięki zastosowaniu skryptów Python możliwe jest asynchroniczne budowanie systemu, dzięki temu mimo zastosowania jedynie modelu SLM na platformie brzegowej, asystent dostarcza interesujących informacji użytkownikowi.

4.4.6. Narzędzia badawcze

Część badawcza pracy wymagała przygotowania środowiska umożliwiającego porównywanie modeli oraz konfiguracji ich pracy. Z tego względu przewidziano wykorzystanie skryptów testowych w języku Python, które pozwalają automatyzować uruchamianie zapytań. Ewaluacja projektu wymagała wykonywania wielu zapytań testowych, niejednokrotnie benchmark trwał ponad dobę, z tego względu powtarzalne i stabilne środowisko testowe zbierające dane było kluczowe. Rozważono zastosowanie narzędzia Optuna, które pozwala systematycznie przeszukiwać przestrzeń parametrów, znajdując optimum w możliwie krótkim czasie [19]. Istotnym elementem oceny jest również jakość odpowiedzi, trudno ją badać deterministycznie, a ręczna ocena każdego zapytania nie była możliwa (bardzo wiele zapytań). Jako rozwiązanie pozwalające automatyzować proces wskazano API dostępnych modeli językowych, wykorzystano serwis OpenRouter, który pozwala korzystać z wielu modeli językowych od znanych operatorów, takich jak Anthropic, Open AI czy Google.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym

Część badawcza stanowi zasadniczy punkt niniejszej pracy magisterskiej. Jej celem było empiryczne sprawdzenie, na ile mały model językowy może działać użytecznie na urządzeniu brzegowym o bardzo ograniczonych zasobach. Raspberry Pi 5 z 8 GB pamięci RAM, bez dedykowanego GPU, wymusza kompromisy między jakością odpowiedzi, opóźnieniem oraz stabilnością inferencji; bez systematycznych pomiarów trudno byłoby uzasadnić wybór modelu, kwantyzacji czy konfiguracji generacji w takim środowisku.

W rozdziale zbadano m.in. wpływ hiperparametrów Ollamy, wariantów kwantyzacji GGUF oraz warunków pomiaru (w tym stanów *cold* i *warm cache*) na metryki jakościowe i wydajnościowe lokalnego SLM. Choć eksperymenty przeprowadzono w kontekście asystenta głosowego systemu Wilga MiLL, uzyskane wnioski mają charakter ogólniejszy: dotyczą uruchamiania małych modeli językowych na typowym sprzęcie IoT i mogą być wykorzystane także w innych aplikacjach brzegowych o podobnych ograniczeniach pamięci i mocy obliczeniowej.

5.1. Architektura i metodologia systemu benchmarkowego

5.1.1. Komponenty systemu testowego

Założeniem dla budowy systemu testowego nie było uruchamianie pełnego produkcyjnego pipeline'u aplikacji Wilga, lecz stworzenie wycinka systemu, który pozwoli poddać testom modele SLM na urządzeniu brzegowym. Architektura systemu pomiarowego składa się z dwóch bloków: serwera orkiestrującego (MacBook) oraz platformy brzegowej (Raspberry Pi 5), przedstawionych na rysunku 5.1.

Po stronie serwera pracuje skrypt `tester.py`, który pełni rolę orchestratora. Dla każdego przypadku z `golden.jsonl` pobiera `system_prompt`, łączy go z pytaniem użytkownika i wysyła żądanie HTTP do Ollamy uruchomionej na Raspberry Pi. Po stronie serwera mierzone są parametry czasowe: TTFT (time to first token) oraz E2E (end to end). Po otrzymaniu odpowiedzi modelu orkiestrator przekazuje ją do modelu zewnętrznego oceniającego jakość odpowiedzi. Ponownie korzystając z OpenRouter podłączono się do modelu Claude, tym razem był to Haiku 3.5. Zastosowano mniejszy, tańszy model, gdyż benchmark wymagał wysyłania wielu żądań, a koszty największych modeli są wygórowane. Haiku zwracał pięć metryk jakościowych, opisanych w tabeli 5.2.

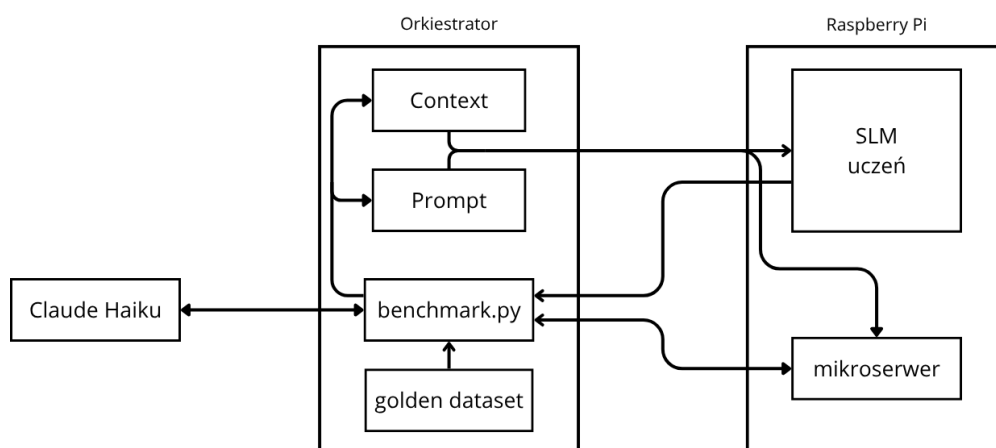
Prompt przekazywany do Haiku składał się z czterech elementów wejściowych:

$$\text{prompt}_{\text{Haiku}} = \{\text{system_prompt}\} + \{\text{question}\} + \{\text{golden_answer}\} + \{\text{model_answer}\}, \quad (1)$$

gdzie `system_prompt` to kontekst stanu domu zbudowany wcześniej z rzeczywistych wierszy danych sensorowych, `question` to pytanie użytkownika, `golden_answer` odpowiedź wzorcowa, natomiast `model_answer` to odpowiedź badanego SLM. W prompcie wskazano ponadto, że kontekst sensorowy jest jedynym źródłem prawdy, a wynik musi zostać zwrócić

jako JSON z pięcioma metrykami. Flaga halucynacji miała być ustawiana tylko wtedy, gdy model podaje konkretne fakty spoza kontekstu. Temperaturę ewaluacji ustawiono na zero, żeby oceny były możliwie powtarzalne. Otrzymana odpowiedź była przetwarzana oraz odrzucana jeśli format nie odpowiadał żądanemu JSON. Wymuszenie pozostaje jednak miękkie, w przypadku dalszej ewaluacji testów za istotne uznano zastosowanie mechanizmów takich jak Pydantic AI czy LangChain pozwalających deterministycznie definiować format odpowiedzi modeli językowych.

Na urządzeniu brzegowym Raspberry Pi 5 działa serwer Ollamy, który dzięki funkcji API pozwala na łatwe odpytywanie wielu modeli zainstalowanych na urządzeniu. Eównolegle działał również mikroserwer `metrics_server.py`, jego wprowadzenie było konieczne aby mierzyć temperaturę CPU, zużycie RAM oraz flagę dławienia (throttling flag, przegrzanie procesora).



Rysunek 5.1. Schemat systemu benchmarkowego.

5.1.2. Zbiór danych testowych

Pracę nad częścią badawczą rozpoczęto od zbioru zawierającego pytania oraz modelowe odpowiedzi. Jest nim plik `golden.jsonl`: stały zbiór 20 przypadków testowych. Każdy rekord zawiera identyfikator przypadku, jego kategorię, tekstowy *system_prompt* zbudowany z realnego wiersza danych sensorowych, pytanie użytkownika oraz wzorcową odpowiedź *golden_answer*. Zbiór wygenerowano jednorazowo przy użyciu modelu Claude Sonnet 4.6. Wybór modelu znacznie większego i bardziej zaawansowanego od lokalnie uruchamianych modeli (omawianych w pracy) pozwolił zapewnić wysoką jakość odpowiedzi referencyjnych. Do integracji i wywołania modelu wykorzystano API platformy OpenRouter, będącej wygodnym agregatorem modeli językowych od wielu dostawców.

Rozkład kategorii przypadków przedstawiono w tabeli 5.1.

Tabela 5.1. Kategorie przypadków w złotym zbiorze testowym (20 przypadków).

Kategoria	Liczba	Co testuje
factual_status	8	Poprawne odczytanie stanu z kontekstu
missing_data	4	Przyznanie braku danych zamiast zgadywania
conflicting_data	3	Wykrycie sprzeczności w danych
stale_data	2	Sygnalizacja nieaktualności odczytu
paraphrase	3	Spójność odpowiedzi na te same pytania
Razem	20	

Kategorie `missing_data` i `stale_data` uznano za najbardziej wymagające, ponieważ są podatne na halucynacje, która jak zaznaczono w analizie literatury 2 jest krytycznym czynnikiem w ocenie jakości odpowiedzi modeli językowych.

5.1.3. Metryki oceny i wzór *composite score*

Tabela 5.2. Metryki oceny stosowane w benchmarku (trzy grupy).

Grupa	Metryka	Zakres	Znaczenie
1	TTFT	ms	Czas do pierwszego tokenu
1	E2E	ms	Czas do ostatniego tokenu
1	<i>generation_ms</i>	ms	E2E – TTFT
1	RAM	MB	Zużycie pamięci na Raspberry Pi
1	temperatura CPU	°C	Temperatura procesora z mikro-serwera
1	flaga dławienia	binarna	Przegrzanie / throttling procesora
2	<i>faithfulness</i>	[0, 1]	Czy każde twierdzenie faktyczne wynika z kontekstu sensorowego
2	<i>answer_relevancy</i>	[0, 1]	Czy odpowiedź adresuje zadane pytanie
2	<i>conciseness</i>	[0, 1]	Czy odpowiedź jest zwięzła, bez zbędnych informacji
2	<i>hallucination_flag</i>	binarna	true, gdy odpowiedź zawiera fakty spoza kontekstu; zeruje wynik łączny
3	<i>polish_quality</i>	[0, 1]	Naturalność i poprawność języka polskiego; rejestrowana, poza <i>composite score</i>

Metryki z tabeli 5.2 podzielono na trzy grupy. Grupa 1 obejmuje metryki wydajnościowe mierzone deterministycznie przez urządzenie brzegowe. Grupa 2 to metryki jakościowe

zwracane przez LLM-oceniający (Haiku 3.5). Grupa 3 obejmuje *polish_quality* został dodany w celu przyszłych ewaluacji jakościowych odpowiedzi.

Metryki z grupy drugiej składają się na *composite score*. W oparciu o literaturę oraz standardy [29], [30], [31] przyjęto go jako sposób matematycznego wyznaczania jakości odpowiedzi porównywanych modeli. Konieczne do wyznaczenia *composite score* było przyjęcie wag dla każdej z metryk. Wzór przedstawiony w równaniu 2.

$$\text{score} = \begin{cases} 0,0 & \text{gdy hallucination_flag} = \text{true} \\ 0,40 \cdot f + 0,30 \cdot r + 0,20 \cdot c + \ell & \text{w przeciwnym razie} \end{cases} \quad (2)$$

gdzie f , r , c to odpowiednio *faithfulness*, *answer_relevancy* i *conciseness*, a $\ell = 0,05 \cdot (1 - \min(e2e_ms/90000, 1)) + 0,05 \cdot (1 - \min(ttft_ms/45000, 1))$.

W prezentowanym rozwiązaniu *faithfulness* ma najwyższą wagę, ponieważ odpowiedź trzymająca się faktów jest niezwykle istotna w kontekście tej pracy- asystenta głosowego. Składniki opóźnienia (E2E i TTFT, łącznie 0,10) zostały rozdzielone, ponieważ moduł TTS może rozpocząć syntezę mowy przed zakończeniem generacji tokenów co powoduje wrażenie bardziej płynnej interakcji.

5.1.4. Dostrajanie parametrów pracy modelu

Optymalizację hiperparametrów inferencji modelu przeprowadzono z wykorzystaniem biblioteki Optuna wykorzystującej metodę TPE (*Tree-structured Parzen Estimator*). Przeszukiwana przestrzeń współczynnik pracy modelu przedstawiono w tabeli 5.3.

Tabela 5.3. Przestrzeń hiperparametrów przeszukiwana przez Optunę (TPE).

Parametr	Typ	Zakres
temperature	float	[0,10, 0,80]
num_ctx	categorical	{1024, 2048, 4096}
num_predict	int	[48, 160]
top_p	float	[0,80, 0,99]
repeat_penalty	float	[1,00, 1,15]

Pełna siatka parametrów wynosi $8 \times 3 \times 3 \times 5 \times 4 = 1440$ kombinacji $\times 20$ promptów = 28800 zapytań. Optuna z 80 trialami (przejściami przez pełny zestaw 20 pytań) ($80 \times 20 = 1600$ ewaluacji LLM-oceniającego) osiąga porównywalne pokrycie obszarów obiecujących dzięki istocie swojego działania. Nie prowadzono testów Nie na wszystkich modelach przeprowadzano testy tej samej długości.

5.1.5. Modele kandydujące

Do badań wybrano sześć modeli klasy SLM, których wielkość zawierała się w przedziale 2–5B parametrów. Kierowano się przede wszystkim wielkością modelu oraz postanowiono postawić na znanych dostawców. Tabela 5.4 przedstawia wybranych kandydatów..

Tabela 5.4. Modele kandydujące i ich udział w benchmarku Optuny.

Model	Parametry	GGUF Q4_K_M	Producent
Llama 3.2 3B	3,2 mld	~2,0 GB	Meta
Phi-3.5 Mini	3,8 mld	~2,4 GB	Microsoft
Gemma 2 2B	2,6 mld	~1,7 GB	Google
Gemma 3 4B	4,0 mld	~2,3 GB	Google
Qwen 2.5 3B	3,0 mld	~2,0 GB	Alibaba
Bielik 4.5B	4,5 mld	~2,7 GB	SpeakLeash

5.2. Etap Optuna: chłodzenie, hiperparametry i dominacja halucynacji

5.2.1. Warunek termiczny pomiaru

Testy rozpoczęto ze wskazaną wyżej grupą modeli. Bardzo szybko okazało się, że czasy odpowiedzi bardzo często przekraczają górną granicę pomiaru, którą wyznaczono na 60 sekund. Po analizie danych jasna stała się przyczyna. W każdym z 500 rekordów procesor pracował w trybie throttling (ang. dławienie). Dzięki pomiarom mikroserwera na RPi możliwe było odczytanie temperatury CPU. Na Raspberry Pi 5 throttling jest metodą ograniczania częstotliwości taktowania procesora w celu obniżenia temperatury, skutkuje ona znacznym obniżeniem mocy obliczeniowej jednostki. Pierwsze ograniczenia aktywowane są przy 80 °C, a twarde dławienie (`vcgencmd get_throttled 0x2`) ok. 85 °C.

Przy przekroczeniu limitu czasu (60 s) Ollama zwraca pustą odpowiedź, którą LLM-judge klasyfikuje jako halucynację (score = 0). Odnotowano 169 timeoutów z korelacją 100% względem flagi halucynacji. Dla gemma3:4b 42% rekordów kończyło się timeoutem. W efekcie ranking z tabeli 5.5 był w zasadzie fałszywy: gemma2:2b zajmował pierwsze miejsce, a lepszy semantycznie gemma3:4b spadał na czwarte – większe modele dłużej obciążają CPU i przy braku chłodzenia generują timeouty. Temperatura CPU utrzymywała się w zakresie 84,0–91,7 °C (mediana 88,9 °C), a throttling występował w 100% przypadków.

Tabela 5.5. Ranking modeli bez aktywnego chłodzenia.

#	Model	Score	Halucynacje	E2E med. [ms]	Mediana temp. [°C]	Throttling
1	gemma2:2b	0,693	16%	37 229	87,8	100%
2	llama3.2:3b	0,589	30%	49 199	88,4	100%
3	qwen2.5:3b	0,547	37%	53 169	88,9	100%
4	gemma3:4b	0,497	42%	57 584	89,8	100%
5	phi3.5	0,024	96%	60 000	90,0	100%

Jako receptę na problemy termiczne wskazano aktywne chłodzenie procesora. Układ RPi został rozbudowany o radiator producenta wyposażony w wentylator. Po instalacji chłodzenia przeprowadzono test sprawdzający czy przyniosło ono zamierony efekt. Flaga throttlingu nie została oznaczona w żadnym z przypadków testowych (0/500), a tempera-

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym

tura CPU do 57,6–73,6 °C (mediana 65,3 °C). Zmiana wprowadziła też pierwsze niezerowe wyniki *composite_score* modeli. Wyniki przedstawiono w tabeli 5.6.

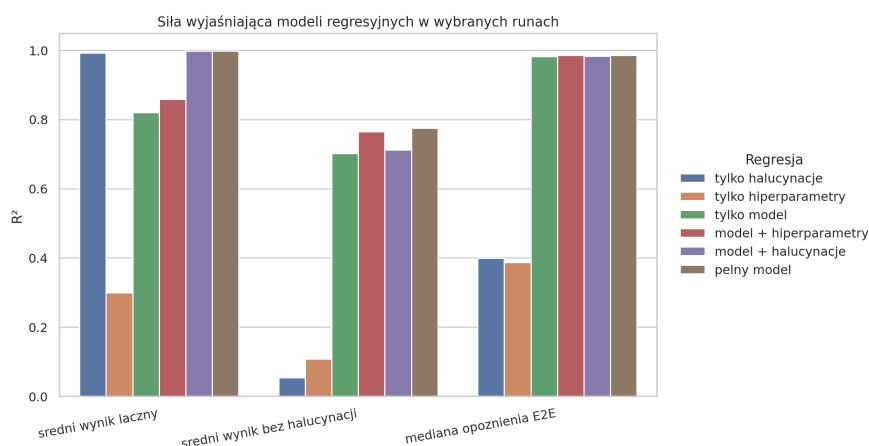
Tabela 5.6. Ranking modeli po zastosowaniu aktywnego chłodzenia.

#	Model	Score	Halucynacje	E2E med. [ms]	Throttling
1	gemma3:4b	0,806	4%	31 742	0%
2	gemma2:2b	0,679	19%	20 879	0%
3	qwen2.5:3b	0,667	19%	29 713	0%
4	llama3.2:3b	0,653	20%	27 145	0%
5	phi3.5	0,380	49%	52 363	0%

Wstępne problemy z regulacją termiczną CPU uwydatniły uwydatnić, jak bardzo stabilność temperaturowa warunkuje ewaluację modeli językowych na urządzeniach klasy Raspberry Pi 5. Nie spotkało się to ze zdziwieniem, gdyż zagadnienie to szeroko opisywane jest w literaturze [3] oraz przez inżynierów w internecie. Aktywne chłodzenie okazało się zapewniać stabilne warunki pracy platformy i umożliwiło rzetelne przeprowadzenie dalszych badań.

5.2.2. Dominacja flagi halucynacji

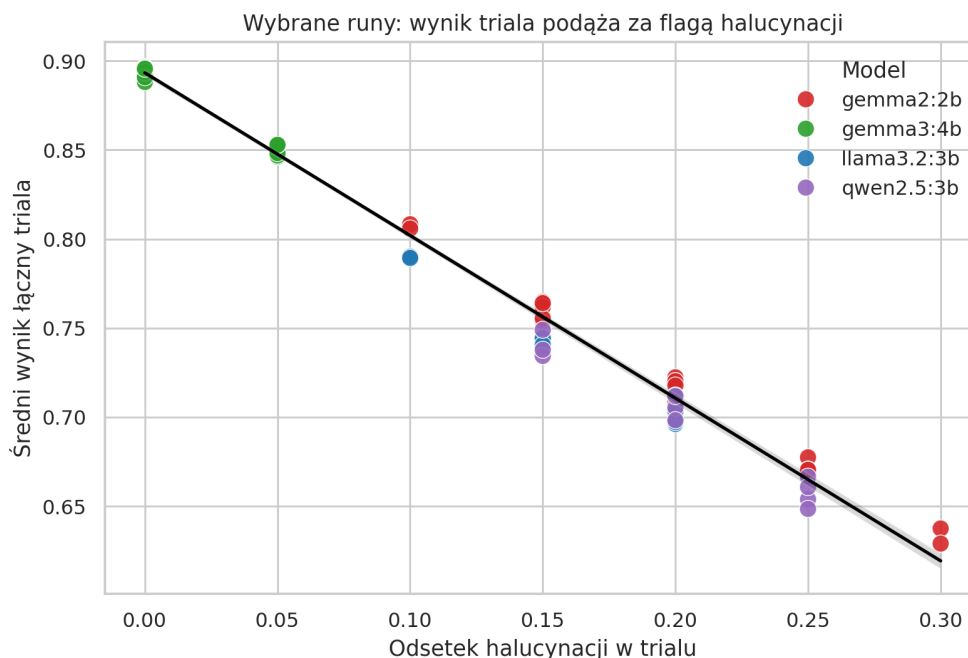
Dzięki aktywnemu chłodzeniu osiągnięto stabilne warunki pracy. Pozwoliło to w stabilnych warunkach przeprowadzić ponownie badania, wykonano przebiegi na wszystkich modelach kandydujących, wymienionych w tabeli 5.4. Po wykonaniu 80 triali (napisz mi tutaj rozkład ile na którym modelu) przystąpiono do wstępnej analizy danych. Jej celem było sprawdzenie czy optymalizacja hiperparametrów wpływa na wartość *composite score*. Przeprowadzono regresję wieloczynnikową, która jest szeroko stosowaną metodą badania istotnych związków między zmiennymi. Wyniki analizy przedstawiono na rysunku 5.2.



Rysunek 5.2. Porównanie R^2 modeli regresji dla *composite score*.

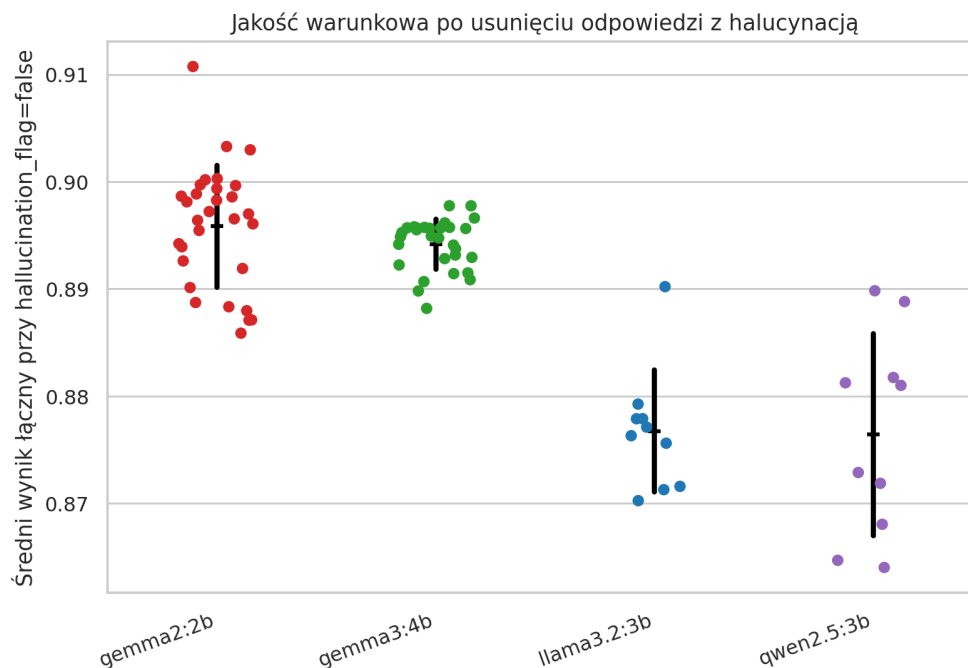
Regresja wieloczynnikowa wykazała, że flaga halucynacji samodzielnie osiąga $R^2 = 0,99$, czyli samodzielnie wyjaśnia ponad 99% wariancji score. Pozostałe zmienne inferencji wyjaśniają jedynie $R^2 = 0,3$, czyli około 30% wariancji score. Wyniki takie wynikają oczywiście

ze sposobu wyznaczenia wzoru na *composite score* 2. Wynik jest zerowany kiedy model halucynuje, jest to celowy zabieg. Jedną z głównych bolączek małych modeli językowych jest ich halucynacja. Tak surowe karanie pozwala łatwo oznaczać przypadki halucynacji. Liniowość zależności *composite score* od wskaźnika halucynacji przedstawiono na rysunku 5.3.



Rysunek 5.3. Zależność *composite score* od wskaźnika halucynacji.

Istotność związku flagi halucynacji z *composite score* jest bardzo wysoka. Nie oznacza to jednak, że inne hiperparametry mają wpływ zerowy. Wykonano analizę ich wpływu na *composite score*. Wyniki przedstawiono na rysunku 5.4.



Rysunek 5.4. Rozpiętość *nonhall_score* i *composite score*.

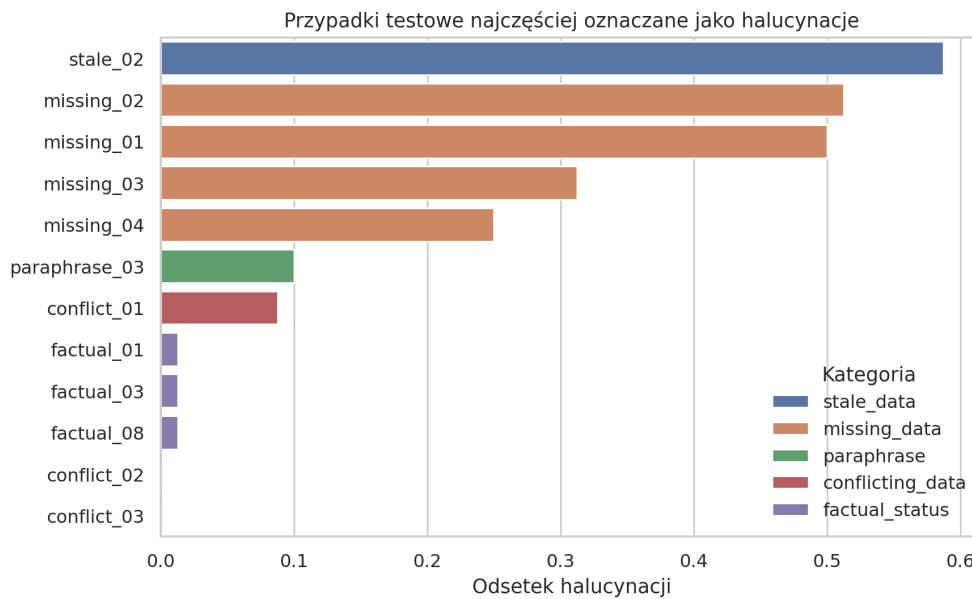
Dzięki wykresowi dobitnie widać, że wyniki jakościowe *po wyłączeniu* halucynacji koncentrują się wokół wąskiego zbioru wartości: rozpiętość wyników po wykluczeniu przypadków halucynacji wynosi zaledwie 0,119, podczas gdy rozpiętość wszystkich przebiegów sięga 0,604 – około pięciokrotnie więcej. Wpływ hiperparametrów na jakość odpowiedzi bez halucynacji jest zatem niewielki, co nie znaczy, że jest on pomijalny. Postanowiono wybrać parametry dające najlepsze wyniki na tym etapie, aby to przy ich zachwaniu prowadzić dalsze badania. Przyjęte wartości przedstawiono w tabeli 5.7.

Tabela 5.7. Hiperparametry inferencji przyjęte do dalszych eksperymentów.

Parametr	Wartość
temperature	0,20
num_ctx	2048
num_predict	128
top_p	0,90
repeat_penalty	1,05

Po dokonaniu analizy wpływu czynników na wynik końcowy miary jakości odpowiedzi. Za zasadne uznano również analizę przypadków testowych. Testy polegają na repetycyjnym zadawaniu serii 20 pytań ze zbioru *golden dataset*. Zbiór ten składa się z pytań kilku kategorii, co opisano w tabeli 5.1. Czynnikiem dominującym wynik jest flaga halucynacji, zbadano zatem jej występowanie w poszczególnych kategoriach. Wyniki przedstawiono na rysunku 5.5.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym

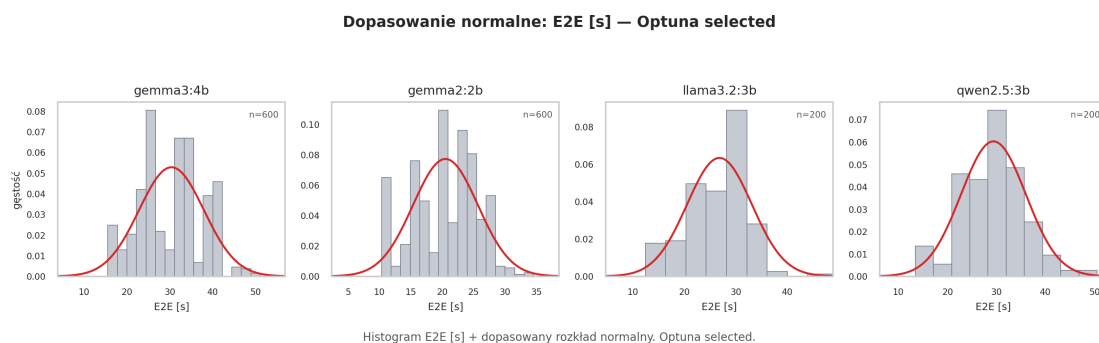


Rysunek 5.5. Wskaźnik halucynacji per przypadek testowy.

Halucynacje koncentrują się w dwóch kategoriach: `missing_data` (39,4%) i `stale_data` (29,4%). Widać wyraźnie, że kiedy zapytanie dotyczy danych niewystępujących lub nieaktualnych w kontekście model chętnie wprowadza do odpowiedzi informacje, które są nieprawdziwe. Jest to ważny wniosek dla kolejnych etapów badań.

5.2.3. Opóźnienie E2E i model Gaussa

Po dokonaniu analizy aspektów jakościowych odpowiedzi postanowiono przyjrzeć się bliżej drugiemu kluczowemu aspektowi asystenta głosowego – czasowi generowania odpowiedzi. Benchmark zbierał już te pomiary (tabela 5.2), więc nie było konieczne przeprowadzanie nowych ewaluacji. W protokole Optuny (zmienny prompt, cold start) czasy E2E układają się w krzywą Gaussa, którą wygodnie opisać średnią oraz odchyleniem standardowym. Histogramy dla poszczególnych modeli przedstawiono na rysunku 5.6.



Rysunek 5.6. Histogram czasów E2E oraz dopasowana krzywa Gaussa per model.

Dla każdego modelu wyliczono średnią $\hat{\mu}$ oraz odchylenie standardowe $\hat{\sigma}$. Przy rozkładzie normalnym przedział $\hat{\mu} \pm \hat{\sigma}$ obejmuje ok. 68% przypadków, a $\hat{\mu} \pm 2\hat{\sigma}$ – ok. 95%. Wyniki zestawiono w tabeli 5.8.

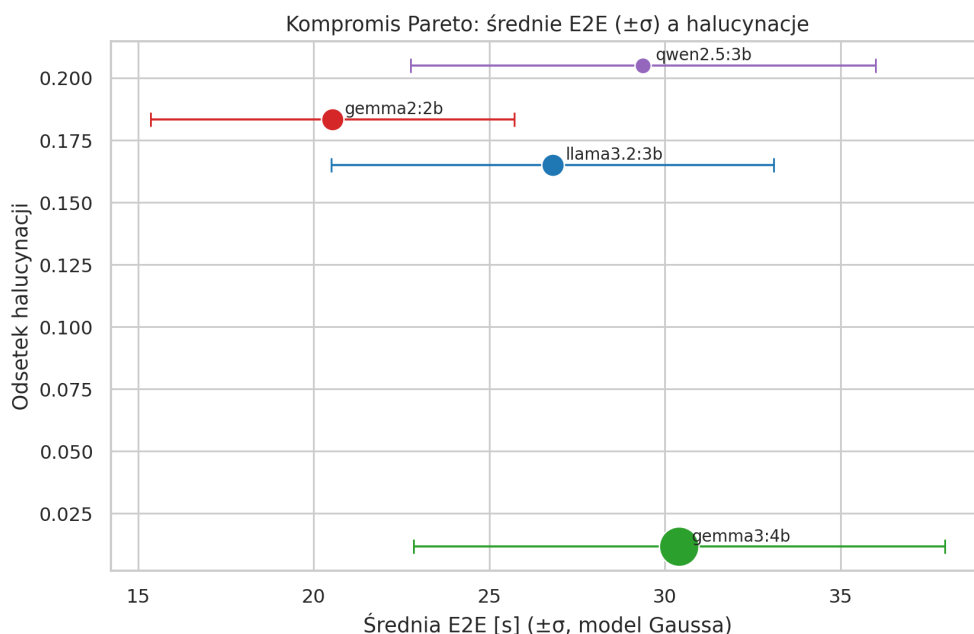
Tabela 5.8. Czasy E2E w protokole Optuny (model Gaussa).

Model	n	$\hat{\mu}$ [s]	$\hat{\sigma}$ [s]	$\hat{\mu} \pm \hat{\sigma}$ [s]	$\hat{\mu} \pm 2\hat{\sigma}$ [s]	CV
gemma2:2b	600	20,5	5,2	15,4–25,7	10,2–30,9	0,25
llama3.2:3b	200	26,8	6,3	20,5–33,1	14,2–39,4	0,24
qwen2.5:3b	200	29,4	6,6	22,8–36,0	16,1–42,6	0,22
gemma3:4b	600	30,4	7,6	22,8–38,0	15,3–45,5	0,25

Z analizy danych wynika, że najszybsze odpowiedzi udziela gemma2:2b – średni czas E2E wynosi 20,5 s, a większość pomiarów ($\hat{\mu} \pm \hat{\sigma}$) mieści się w przedziale 15,4–25,7 s. Pozostałe modele są wyraźnie wolniejsze: llama3.2:3b ok. 26,8 s, qwen2.5:3b ok. 29,4 s, a gemma3:4b ok. 30,4 s. Odchylenie stanowi u wszystkich ok. jednej czwartej średniej (CV 0,22–0,25), więc modele różnią się przede wszystkim średnim czasem odpowiedzi, a nie rozrzutem między przebiegami.

Sam czas nie może być jednak interpretowany samodzielnie. Istotny jest kompromis między jakością a opóźnieniem. llama3.2:3b i qwen2.5:3b mają E2E zbliżone do gemma3:4b, lecz halucynują w 16–21% przypadków, podczas gdy gemma3:4b jedynie w ok. 1,2%. gemma2:2b jest najszybsza, ale halucynuje w ok. 18% odpowiedzi. Na rysunku 5.7 widać ten układ: żaden model nie jest jednocześnie najszybszy i najmniej halucynujący.

Dla asystenta informującego o stanie domu błędna informacja jest groźniejsza niż kilka sekund dłuższego czekania. Dlatego do kolejnych etapów optymalizacji opóźnienia wybrano gemma3:4b. Rozsądny zestaw hiperparametrów z tabeli 5.7 wystarcza; Optuna nie wskazała konfiguracji wyraźnie lepszej jakościowo.



Optuna results-selected-9.05. Oś X: $\text{mean} \pm \sigma$ (Gauss), nie mediana. Rozmiar punktu $\sim \text{score_mean}$.

Rysunek 5.7. Kompromis Pareto: opóźnienie E2E a wskaźnik halucynacji.

5.3. Etap kwantyzacji: cold vs warm i model log-normalny

5.3.1. Typy kwantyzacji GGUF

Formatem dla zapisu modeli wspieranym przez aplikację Ollama jest GGUF. Ten sposób zapisu pliku pozwala na różne poziomy kwantyzacji wag. Jak wykazano w rozdziale 2.2, kwantyzacja wag pozwala na znaczne przyspieszenie generowania odpowiedzi. Przystępując do W badaniach wpływu zmiany precyzji wag modelu uwzględniono cztery warianty:

- **Q8_0** – 8-bitowe wagi bez grupowania; baseline,
- **Q4_K_M** – 4-bitowe wagi z grupowaniem K i mixed precision M,
- **Q4_0** – 4-bitowe wagi bez grupowania,
- **Q2_K** – 2-bitowe wagi z grupowaniem; agresywna kompresja.

Mniejsza precyzja (liczba bitów) oznacza mniejszy plik i co za tym idzie mniej danych do przenoszenia do pamięci podręcznej w celu ewaluacji modelu. Literatura mówi o pozytywnym wpływie kwantyzacji na czas generowania odpowiedzi. Należy pamiętać, że zbyt agresywna kwantyzacja może destabilizować generację. Ponownie optymalizacja w tym elemencie opiera się na pewnego rodzaju balansie (trade off) między zmniejszeniem wielkości modelu, a zachowaniem odpowiedniej jakości.

Na potrzeby testów dokonano kwantyzacji trzech modeli językowych (gemma3-4b, gemma4-e2b, bielik-4.5b). Modele wybrano na podstawie poprzednich etapów badań, Rodzina Gemma od Google wykazywała najlepsze wyniki oraz odnotowano w nich wpływ na metody optymalizacji. Bielik natomiast w poprzednich etapach badań był bardzo wolny, dlatego postanowiono sprawdzić jego zachowanie przy zmianie precyzji wag. Tabela 5.9 przedstawia rozmiary plików wyjściowych GGUF wraz z ich rozmiarem.

Tabela 5.9. Rozmiary plików GGUF [GB] dla badanych modeli.

Model	Q2_K	Q4_0	Q4_K_M	Q8_0
gemma3-4b	1,61	2,20	2,32	3,85
gemma4-e2b	2,78	3,13	3,19	4,63
bielik-4.5b	1,65	2,52	2,68	4,71

5.3.2. Wstępne pomiary E2E i mechanizm cold vs warm cache

Analiza literatury pozwalała zakładać, że obniżenie precyzji wag przyspiesza inferencję, ponieważ głównym ograniczeniem jest ograniczone przepustowością pamięci [9]. Pierwsze porównanie wariantów GGUF na metryce E2E tego nie potwierdzało. Modele przez wiele ewaluacji testowych nie wykazywały wyraźnego przyspieszenia przy przejściu z Q8_0 (8 bitów) do Q4 (4 bity): warianty Q4_0 i Q4_K_M dla gemma3-4b osiągały praktycznie identyczne czasy. Tabela 5.10 zestawia mediany modelu log-normalnego e^{μ} całkowitego czasu odpowiedzi ($n \approx 100$), na tym etapie nie rozróżniano czasu całkowitego oraz TTFT.

Tabela 5.10. Wstępne porównanie wariantów kwantyzacji (E2E).

Model	Q2_K	Q4_0	Q4_K_M	Q8_0
gemma3-4b	53,9 s	38,5 s	37,9 s	71,2 s
gemma4-e2b	43,8 s	29,9 s	26,7 s	82,8 s
bielik-4.5b	94,0 s	87,9 s	86,6 s	160,8 s

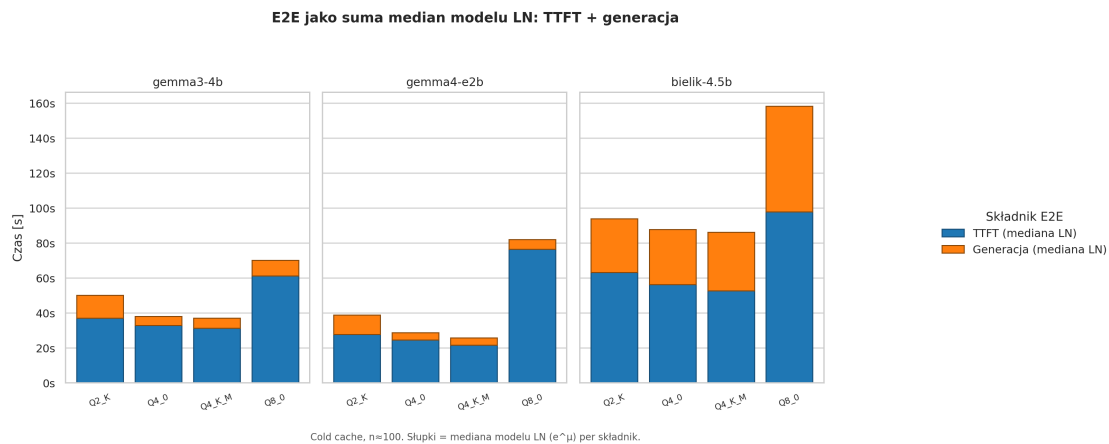
Dla obu modeli Gemma Q4_0 i Q4_K_M różnią się niewiele, założono, że różnica wynika z błędów pomiarowych i czynników losowych. Q8_0 jest wyraźnie wolniejszy, lecz nie w sposób, jakiego oczekiwano po samej kwantyzacji: albowiem różnica nie układa się proporcjonalnie do precyzji wag. Wariant Q2_K wypada gorzej niż Q4 mimo najmniejszego pliku modelu GGUF; anomalie tę omówiono w sekcji 5.3.5.

Poszukiwanie przyczyny rozpoczęto od analizy literatury. Zagłębienie się w artykuły naukowe oraz dokumentacje Ollamy rzuciło nowe światło na problem. Pope et al. [32] wprowadzili rozróżnienie dwóch faz: *prefill* (równoległe przetworzenie całego promptu oraz zbudowanie pamięci, KV-cache) oraz *decode* (autoregresyjne generowanie kolejnych tokenów). Czas do pierwszego tokenu (TTFT) jest zdominowany przez prefill, natomiast kwantyzacja wag wpływa przede wszystkim na fazę dekodowania, czyli samo generowania tokenów wyjściowych przez model. Faza ta jest ograniczona przede wszystkim przepustowością pamięci [9]. Kwon et al. [33] pokazali, że KV-cache jest kluczową strukturą serwowania LLM: jej ponowne użycie pozwala pominąć ponowne liczenie prefillu, przy założeniu, że prompt rozpoczyna się w ten sam sposób (prefix). Ollamie usuwa model z pamięci RAM po okresie bezczynności, czas ten ustawiany jest przez parametr `keep_alive` (domyślna wartość to 5 minut) [34]. Silnik llama.cpp, na którym opiera się Ollama, udostępnia analogiczny mechanizm ponownego użycia prefiksu KV (`cache_prompt`).

Wysunięto wnioski, mówiące o tym, że pierwsze pomiary były zdominowane przez czas prefill. Zjawisko to nazwano *cold cache*. Model przy każdym zapytaniu był ładowany z dysku do pamięci RAM. Następnie urządzenie wykonywało pełny prefill. Metryka E2E była więc sumą długiego TTFT i krótkiej generacji tokenów wyjściowych, $E2E = TTFT + generation_ms$, stąd różnice były niewielkie.

Rysunek 5.8 pokazuje dekompozycję E2E dla cold-cache ($n \approx 100$ per komórka model $\times$ wariant). Wysokości słupków to mediany modelu log-normalnego $e^{\hat{\mu}}$.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym



Rysunek 5.8. Dekompozycja E2E na TTFT i *generation_ms* (cold cache).

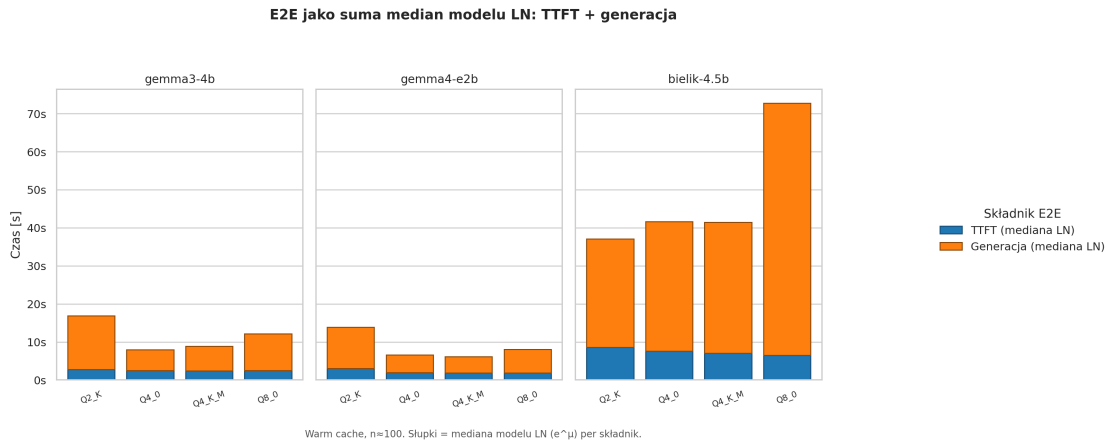
Na wykresie doskonale widać, że TTFT jest dominującym składnikiem sumy czasu generacji. Zauważono, że jest niezależny od jej typu. Dla gemma3-4b/Q4_K_M mediana LN wynosi ok. 31,3 s TTFT przy 5,8 s generacji oraz 37,9 s E2E zatem prefill to ok. 83% całkowitego opóźnienia odpowiedzi modelu. Warianty Q4_0 i Q4_K_M wykazały odpowiednio: TTFT 33,0 s oraz 31,3 s, a czas generacji 5,0 s oraz 5,8 s. Kwantyzacja powoduje różnicę rzędu sekundy przy trzydziestosekundowym prefillu. Wynik Q8_0 to TTFT ok. 61,3 s, co wynika z większego pliku GGUF.

Na otrzymane wyniki wpływ miała oczywiście również architektura Raspberry Pi 5. Urządzenie to nie dysponuje GPU pozwalającym przyspieszać czas inferencji modeli LLM, całość obliczeń wykonuje CPU ARM Cortex-A76. Oficjalna dokumentacja procesora BCM2712 wskazuje, że sprzęt posiada 32-bitowy interfejs LPDDR4X o przepustowości do 17 GB/s [35]. Proces prefill'u wymaga jednorazowego przetworzenia całego promptu i zbudowania KV-cache [32]. Faza dekodowania (generowania tokenów wyjściowych), w której przy każdym tokenie trzeba ponownie odczytać wagi z pamięci [9], stanowi w tym protokole zaledwie ok. 13–15% E2E, więc zysk z niższej precyzji bitowej okazuje się być niewidoczny.

Aby wyizolować czas generacji, zastosowano rozwiązanie nazwane *warm-cache*: stały *system_prompt*, *keep_alive=30m*, *streaming=True* oraz zaczęto wyliczać metrykę *generation_ms = e2e_ms - ttft_ms* per rekord testowy. Model i KV-cache pozostają wówczas w RAM między zapytaniem, a prefill ogranicza się do nowego sufiksu. Aby osiągnąć zysk konieczne jest zachowanie stałej konstrukcji promptu: KV-cache jest wykorzystywany tylko wtedy, gdy tokeny wejściowe od początku wiadomości są identyczne. Szerszy opis tego mechanizmu oraz wynikającej z niego architektury promptu zawarto w sekcji 5.4 (Prompt engineering i prefix caching).

Rysunek 5.9 pokazuje dekompozycję czasu opóźnienia odpowiedzi modelu po przejściu na rozwiązanie typu *warm-cache*.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym



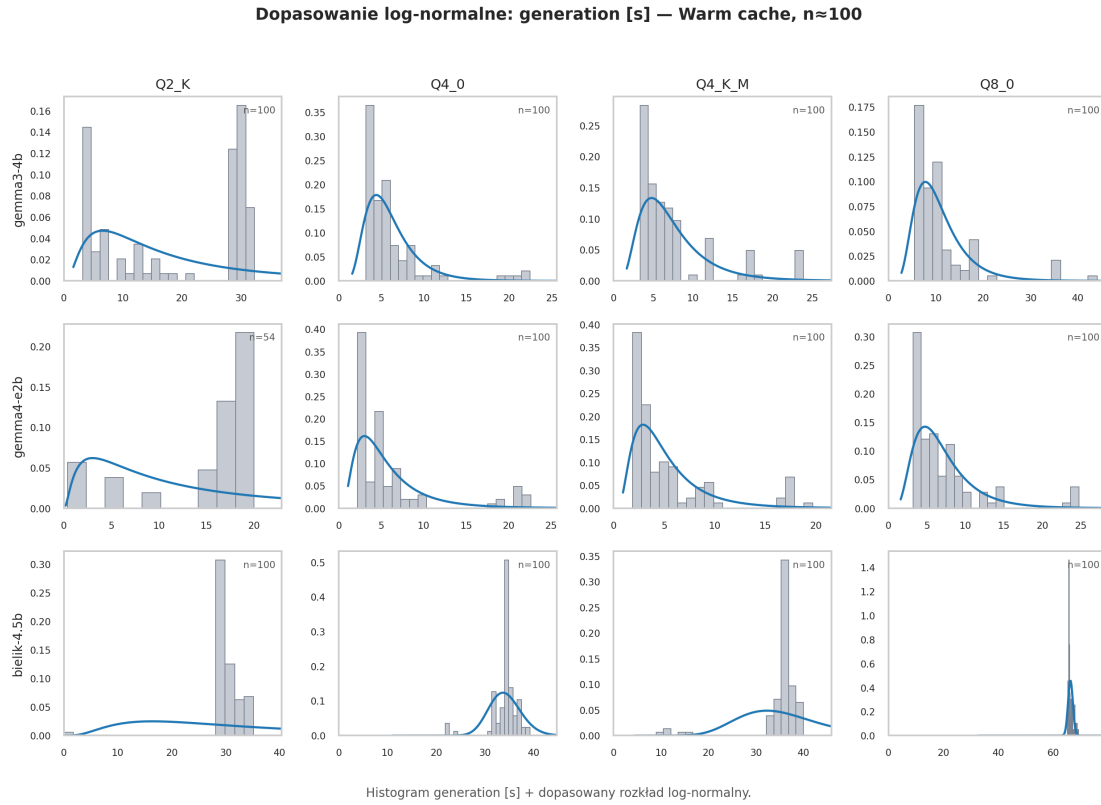
Rysunek 5.9. Dekompozycja E2E na TTFT i *generation_ms* (warm cache).

Zmiana podejścia daje ogromną poprawę. Dla gemma3-4b/Q4_K_M TTFT spada z ok. 31,3 s do ok. 2,44 s (redukcja o 92%), a całkowity czas odpowiedzi E2E z ok. 37,9 s do ok. 9,0 s (redukcja o 76%). Udział czasu generacji (*generation_ms*) w E2E rośnie z ok. 15% do ok. 71%, więc wpływ kwantyzacji zaczyna być widoczny, nie jest już maskowany przez prefill. Q4_K_M osiąga 6,42 s generacji, a Q8_0 9,66 s, czyli przyspieszenie wywołane kwantyzacją to 1,50×.

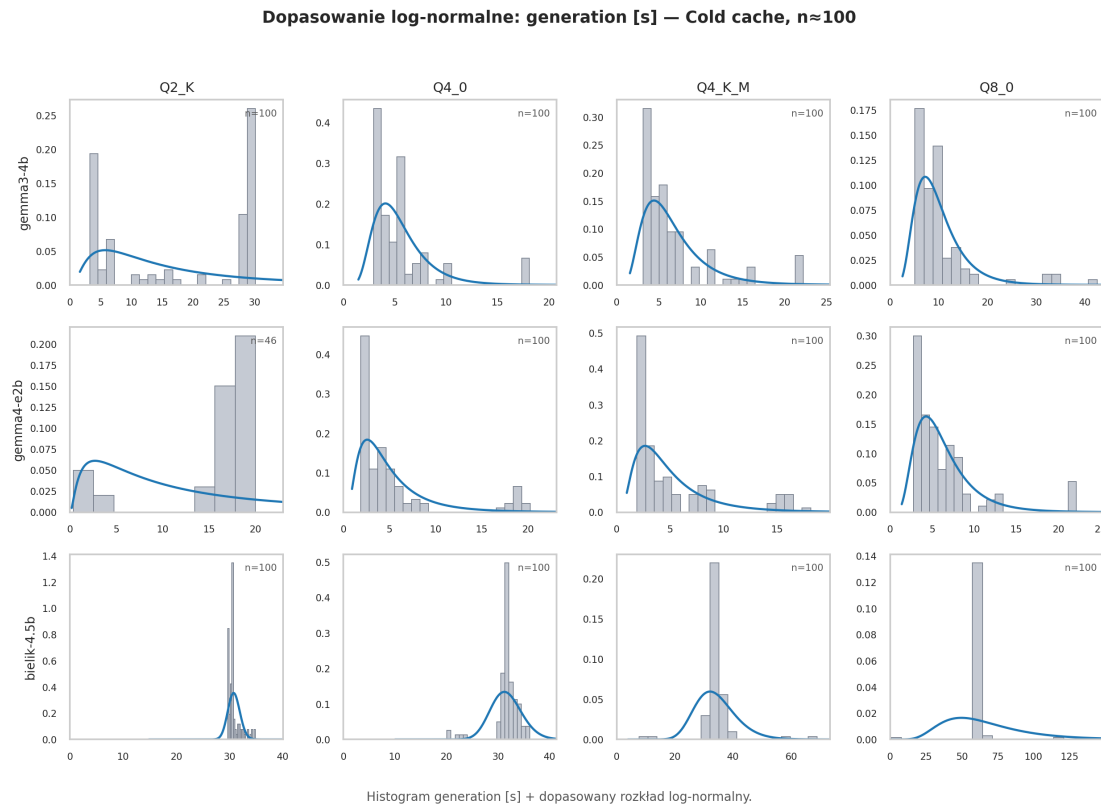
5.3.3. Model log-normalny dla czasów generacji

Aby dobitnie wykazać wpływ kwantyzacji na opóźnienie czasu odpowiedzi modelu, przeprowadzono matematyczną analizę wyników. Czasy generacji są dodatnie i prawostronnie skośne (zdarzają się pojedyncze zdecydowanie dłuższe wyniki czasowe). Przyjęto więc wspólny model roboczy log-normalny: lokalizacja $e^{\hat{\mu}}$ (mediana modelu LN), przedział $e^{\hat{\mu} \pm \hat{\sigma}}$. Dwa poniższe rysunki 5.10 oraz 5.11) przedstawiają histogramy *generation* oraz dopasowaną gęstość LN odpowiednio dla przypadków *warm-cache* oraz *cold-cache*.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym



Rysunek 5.10. Dopasowanie rozkładu log-normalnego do *generation_ms* (warm cache).



Rysunek 5.11. Dopasowanie rozkładu log-normalnego do *generation_ms* (cold cache).

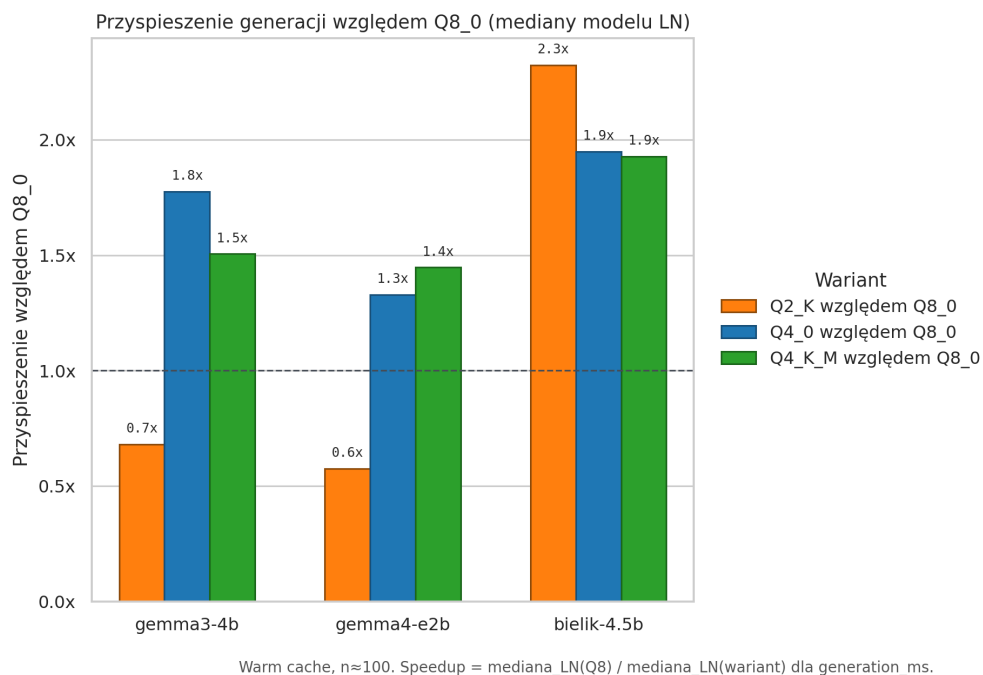
5.3.4. Wyniki warm-cache

Tabela 5.11 zestawia mediany modelu LN dla TTFT, generacji i E2E oraz przyspieszenie generacji względem Q8_0 ($\text{speedup} = e^{\hat{\mu}_{Q8}} / e^{\hat{\mu}_{\text{wariant}}}$).

Tabela 5.11. Wyniki kwantyzacji przy warm cache.

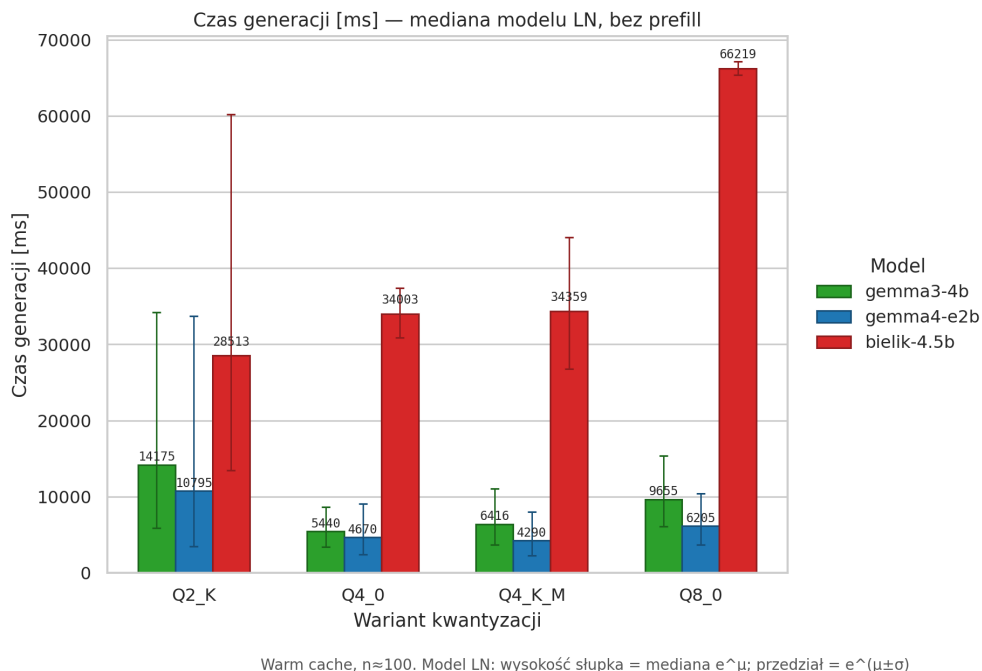
Model	Wariant	GGUF	TTFT	Gen	E2E	Speedup
gemma3-4b	Q2_K	1,61G	2,74s	14,18s	17,74s	0,68×
gemma3-4b	Q4_0	2,20G	2,52s	5,44s	8,06s	1,77×
gemma3-4b	Q4_K_M	2,32G	2,44s	6,42s	9,02s	1,50×
gemma3-4b	Q8_0	3,85G	2,51s	9,66s	12,30s	1,00×
gemma4-e2b	Q2_K	2,78G	3,08s	10,79s	16,44s	0,57×
gemma4-e2b	Q4_0	3,13G	1,93s	4,67s	6,80s	1,33×
gemma4-e2b	Q4_K_M	3,19G	1,83s	4,29s	6,30s	1,45×
gemma4-e2b	Q8_0	4,63G	1,88s	6,20s	8,22s	1,00×
bielik-4.5b	Q2_K	1,65G	8,58s	28,51s	39,19s	2,32×
bielik-4.5b	Q4_0	2,52G	7,59s	34,00s	41,64s	1,95×
bielik-4.5b	Q4_K_M	2,68G	7,06s	34,36s	41,67s	1,93×
bielik-4.5b	Q8_0	4,71G	6,54s	66,22s	72,78s	1,00×

Rysunek 5.12 przedstawia zyski w sposób graficzny: przyspieszenie czasu *generation_ms* (E2E – TTFT) względem wariantu podstawowego Q8_0 (modelu 8-bitowego). Modele z rodziny Gemma w wariantach Q4 notują przyspieszenie (1,33–1,77×), Q2_K wypada poniżej 1×, co oznacza spowolnienie. Bielik przyspiesza we wszystkich wariantach, najmocniej wariant Q2_K (2,32×).



Rysunek 5.12. Przyspieszenie *generation_ms* względem Q8_0 (warm cache).

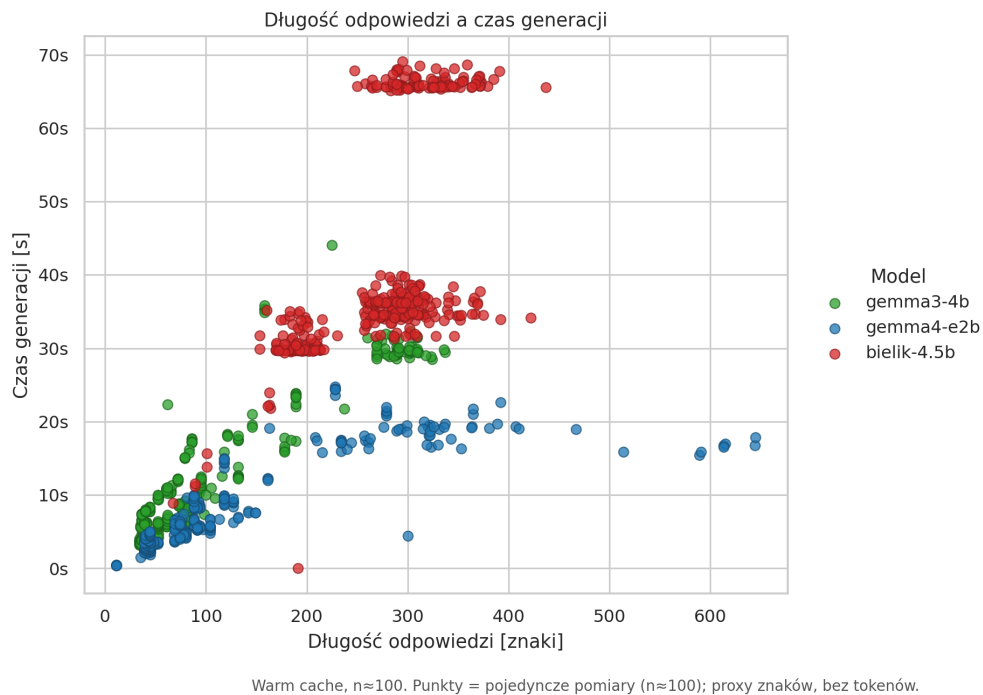
Skupiono się również na analizie samych wyników czasowych. Rysunek 5.13 podaje medianę modelu LN $e^{\hat{\mu}}$; pionowe odcinki to przedział $e^{\hat{\mu} \pm \hat{\sigma}}$ (rozrzut rozkładu LN, ok. 68% masy). Bielik przy Q4_0 i Q8_0 jest bardzo powtarzalny (wąski odcinek, przy Q8_0 ok. 65–67 s), przy Q2_K odcinek jest znacznie dłuższy dla wszystkich modeli (dla gemma3-4b ok. 65–67 s), przy Q2_K odcinek jest znacznie dłuższy dla wszystkich modeli (dla gemma3-4b ok. 6–34 s). Dla modeli z rodziny Gemma kwantyzacja 2-bitowa Q2_K nie skraca opóźnienia generacji, przeciwnie wydłuża ją. Wskazuje to na utratę zwięzłości, omówioną w sekcji 5.3.5.



Rysunek 5.13. Czas *generation_ms* według wariantu kwantyzacji.

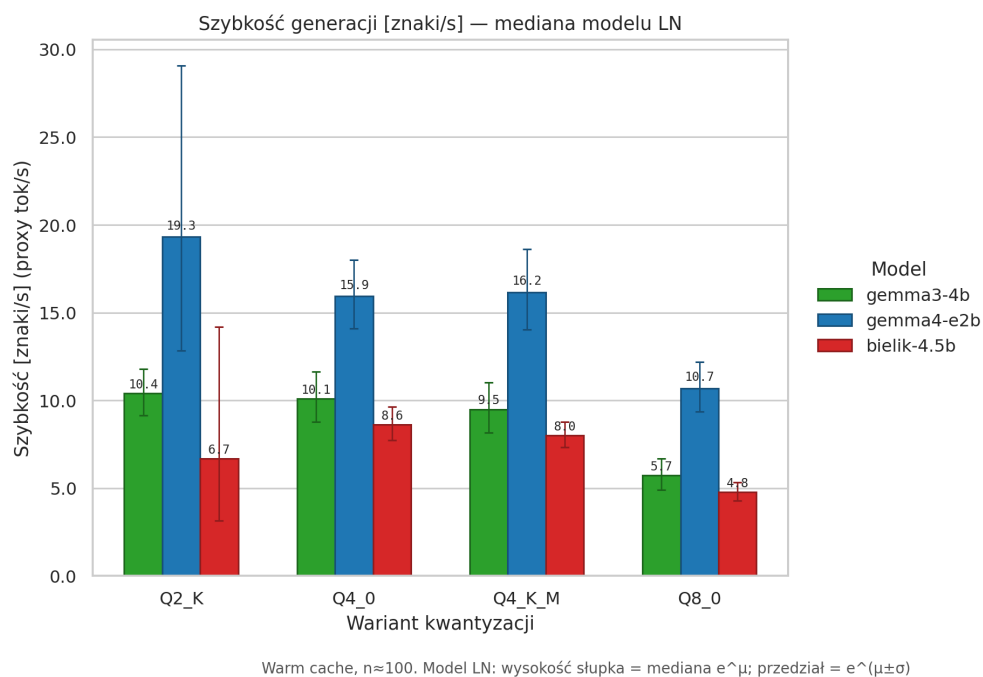
Czas, po którym wczasowicz otrzyma odpowiedź, zależy także od jej długości. Modele oraz ich warianty kwantyzacji charakteryzują się różną zwięzłością (małe modele przestają rozumieć zadanie). Rysunek 5.14 ilustruje to zjawisko: każdy punkt to pojedynczy przebieg (różne poziomy kwantyzacji, kolor oznacza model). Czas rośnie wraz z liczbą tokenów wyjściowych, ale charakterystyka jest różna.

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym



Rysunek 5.14. Czas generacji a długość odpowiedzi.

Samo opóźnienie generacji nie wystarcza więc do oceny kwantyzacji. Rysunek 5.15 pokazuje więc szybkość generacji [znaki/s], liczoną per rekord jako $\text{answer_chars}/(\text{generation_ms}/1000)$. Modele poddane mocniejszej kwantyzacji (widać to na przykładzie Gemmy Q2_K) wykazują zbliżoną lub wyższą przepustowość znaków niż Q4 (ok. 10 zn/s u gemma3-4b), a mimo to dłuższy czas na rysunku 5.13, wynika to z faktu, że odpowiedzi są po prostu dłuższe.



Rysunek 5.15. Szybkość generacji według wariantu kwantyzacji.

5.3.5. Anomalie i wnioski

Wariant gemma3-4b/Q2_K charakteryzuje się długim czasem generacji (E2E) mimo najmniejszego pliku GGUF (1,61 GB, mediana LN ok. 14 s przy 6,42 s dla wariantu Q4_K_M). Przy tak niskiej precyzji wag model przestaje rozumieć postawione przed nim zadanie i gubi zwężłość: mediana długości odpowiedzi rośnie z ok. 53–58 znaków dla wariantów Q4/Q8 do ok. 270 znaków (ok. 4× dłuższa odpowiedź). Na rysunku 5.14 widać to jako ogon zielonych punktów gemma3-4b przy 250–350 znakach.

Ciekawe zachowanie wykazuje model gemma4-e2b dla kwantyzacji Q2_K. Dla tego modelu zauważono zdecydowane spadki jakości. W pomiarach testowych ponad połowa przebiegów kończy się pustą odpowiedzią (11 z 20), a odsetek trafień cache (*warm_cache*) spada do 0,45. Podejrzany winowajcą tego zjawiska jest architektura modelu. Gemma 4 jest modelem MoE (ang. *Mixture-of-Experts*), agresywna kwantyzacja 2-bitowa może zaburzać routing między ekspertami i w literaturze wiąże się z gwałtownym spadkiem jakości (rozdział 2.2).

Podjęto wiele prób testów polskiego modelu Bielik. Skupiono się głównie na bielik-4.5b, który jest modelem SLM. Bielik reagował responsywnie na stosowane metody ulepszania jego performance'u, przyspiesza po kwantyzacji (nawet 2,32× przy Q2_K). Mimo przyspieszenia, model pozostawał zdecydowanie zbyt wolny by rokować jako dobry kandydat na bazę asystenta głosowego. Dla kwantyzacji Q4_K_M mediana LN generacji wynosi ok. 34 s, około 5× więcej niż 6,42 s u gemma3-4b/Q4_K_M. Przy E2E rzędu 42 s model odpada z konfiguracji produkcyjnej na Raspberry Pi 5.

Jako najlepsi kandydaci jawiły się modele autorstwa Google. W rodzinie Gemma oba warianty 4-bitowe: Q4_0 (bez grupowania) i Q4_K_M (różniące się schematem grupowania, omówione w sekcji 2.2) dają czytelne przyspieszenie względem Q8_0 bez załamania zwężłości charakterystycznego dla Q2_K, a różnica między nimi jest zauważalnie mniejsza niż skok z ośmiu do czterech bitów: u gemma3-4b szybszy jest Q4_0 (1,77×, 5,44 s generacji wobec 1,50× i 6,42 s przy Q4_K_M), dla modelu gemma4-e2b wygranym jest wariant Q4_K_M (1,45× wobec 1,33×), przy niemal identycznym rozmiarze GGUF. Kwantyzacja nie pozwala zredukować halucynacji, zmniejszenie precyzji może pozytywnie wpływać na czas generacji. Zbyt duża kwantyzacja natomiast oczywiście powoduje obniżenie jakości odpowiedzi i może wywołać halucynacje. Dalsze obniżanie precyzji poniżej 4 bitów nie skraca opóźnień bez utraty jakości, więc kolejnym krokiem jest redukcja TTFT przez układanie promptu pod *prefix caching*.

5.4. Prompt engineering i prefix caching

5.4.1. Prefix caching promptu systemowego

W architekturze transformerowej pary klucz–wartość (ang. *key-value*) dla tokenów promptu są wyliczane w fazie *prefill* i zapisywane w KV-cache. Cache pozostaje ważny tylko wtedy, gdy tokeny od początku sekwencji są identyczne między zapytaniem – kolejna tura nie wymaga wtedy pełnego prefillu. Przy *cold cache* TTFT stanowi większość E2E; przy stałym prefiksie i *warm cache* TTFT spada o rząd wielkości (sekcja 5.3).

5. Część badawcza: analiza porównawcza i optymalizacja SLM na urządzeniu brzegowym

Dla architektury Teacher–Student (rozdział 3.2) oznacza to dwa założenia: blok stały (reguły, persona) musi być prefiksem, a blok zmienny (odczyty, insight Nauczyciela, historia, pytanie) – sufiksem.

5.4.2. Budowa promptu systemowego

Prompt asystenta głosowego Wilgi jest długi względem okna kontekstu modeli SLM. Optuna przeszukiwała `num_ctx` w zbiorze {1024, 2048, 4096}; do dalszych eksperymentów przyjęto 2048 tokenów przy `num_predict` równym 128 (tabela 5.7). Prefiks systemowy liczy ok. 1900 znaków (szesnaście zdań). Część zmienna to lista będąca bezpośrednim odczytem z bazy danych w formacie – pomieszczenie: czujnik – metryka = wartość (np. – Łazienka: climate sensor – temperatura = 15,4 °C). Ta część składa się z 63 pól; puste komórki są usuwane, więc 63 to wartość maksymalna. Wielkość tej części to maksymalnie 2400 znaków. Findings Nauczyciela to 1–4 krótkie zdania w języku naturalnym na temat anomalii w systemie (ok. 150–400 znaków). Prompt wzbogacany jest również o historię, czyli ostatnie sześć wiadomości, typowo trzy pary pytanie–odpowiedź: ok. 300 znaków.

Dla języka polskiego można przyjąć rząd wielkości ok. 2,5–3,5 znaku na token, więc prefiks to ok. 750 tokenów, fakty 800, Findings 50–150, a historia ok. 100. Sam prefiks i fakty ($750 + 800 = 1550$) zajmują już istotną część okna 2048; z Findings i historią ($1550 + 150 + 100 = 1800$) maksymalna wartość to 1800 tokenów, do której doczepione zostaje obecne pytanie użytkownika. Wartość 1024 jest oczywiście wobec tej konstrukcji zbyt ciasna. Okno 4096 dawałoby zapas, ale wydłuża prefill i zwiększa zużycie RAM na Pi. Dobrą praktyką jest ograniczanie okna kontekstu do wartości minimalnej pozwalającej spełnić założenia architektury promptu. Zdecydowano się zatem na 2048 tokenów.

Złożono więc prompt jako prefiks (stały) i sufiks (zmienny), w kolejności z tabeli 5.12. Podział odpowiada założeniu KV-cache: identyczny początek sekwencji, zmienne elementy wyłącznie na końcu.

Tabela 5.12. Struktura promptu Ucznia: prefiks, sufiks, typ i długość.

Warstwa	Składnik	Typ	Znaki / tokeny	Znaczenie
Prefiks	persona i reguły (system)	stała	1900 / 750	Wilguś, język, cytowanie wyłącznie z faktów, transkrypcja, zakaz zgadywania
Sufiks	fakty CSV (user)	zmienna	do 2400 / 800	lista odczytów; pinowana na czas sesji
Sufiks	Findings Nauczyciela	zmienna	150–400 / 50–150	1–4 zdania o anomaliiach; pomijane gdy brak
Sufiks	historia (6 wiad.)	zmienna	300 / 100	trzy tury pytanie–odpowiedź
Sufiks	bieżące pytanie (user)	zmienna	transkrypcja	zawsze ostatni element sekwencji

W pracy nie znaleziono przestrzeni na dalsze testy treści promptu, w tym na zmianę części stałej (skrócenie reguł, inna persona lub kolejność zdań). Krótszy prefiks w teorii

mógłby skrócić prefill, czym pozytywnie wpłynąłby na czas generacji w przypadku *cold cache*. Pozostaje to ciekawym kierunkiem dalszych badań.

5.5. Analiza wyników i rekomendacja produkcyjna

5.5.1. Podsumowanie badań

Część badawcza rozbita była na trzy kolejne fazy, ich sekwencyjność była wynikiem postępującego procesu badawczego, nie była narzucona z góry. Pracę rozpoczęto od strojenia hiperparametrów inferencji modelu (parametry aplikacji Optuna) - ten etap w głównej mierze skupiony był na jakości odpowiedzi, stanowiła ona główny cel badań. Opóźnienie oczywiście również rejestrowano, by odrzucić konfiguracje zdecydowanie zbyt wolne, ale nie ono było tu osią optymalizacji. Następnie zbadano wpływ kwantyzacji wag plików GGUF na czas generacji, to tutaj opóźnienie wyszło na pierwszy plan. Na końcu ułożono prompt tak, aby stały prefiks mógł być ponownie wykorzystywany przez mechanizm *prefix caching*.

Po ustabilizowaniu temperatury procesora dzięki zastosowaniu aktywnego chłodzenia porównano modele optymalizując parametry ich pracy, na co pozwala API Optuny. Jak się okazało, o końcowych wynikach decydowała przede wszystkim flaga halucynacji, a nie zestaw hiperparametrów i ich optymalizacja (szerzej opisano w sekcji 5.2). Czasy E2E w tym protokole (zmienny prompt, *cold start*) układają się w krzywą Gaussa, zatem wyniki wygodnie opisać średnią $\hat{\mu}$ oraz odchyleniem $\hat{\sigma}$. Wyniki zestawiono w tabeli 5.13.

Tabela 5.13. Wyniki fazy Optuny według modelu.

Model	Score	Hal.	$\hat{\mu}$ [s]	$\hat{\sigma}$ [s]	$\hat{\mu} \pm \hat{\sigma}$ [s]
gemma3:4b	0,884	1,17%	30,4	7,6	22,8–38,0
llama3.2:3b	0,732	16,5%	26,8	6,3	20,5–33,1
gemma2:2b	0,732	18,3%	20,5	5,2	15,4–25,7
qwen2.5:3b	0,697	20,5%	29,4	6,6	22,8–36,0

Analiza pokazuje, że gemma3:4b jest najwolniejszym modelem- średni czas E2E wynosi ok. 30,4 s, lecz halucynuje zaledwie w 1,17% odpowiedzi, około czternastokrotnie rzadziej niż pozostałe modele. Pokazuje to tak zwany trade-off między jakością, a opóźnieniem. gemma2:2b odpowiada najszybciej (ok. 20,5 s), ale wskaźnik halucynacji sięga aż 18,3%. llama3.2:3b i qwen2.5:3b mają czas E2E zbliżony do zwycięzcy, przy 16–21% halucynacji. Model phi3.5 został usunięty z badań już po drugim przebiegu: nawet po zastosowaniu aktywnego chłodzenia bardzo niskie wyniki (score 0,380) i halucynował w aż 49% przypadków (tabela 5.6). Dla asystenta informującego o stanie domu błędna informacja jest groźniejsza niż kilka sekund więcej oczekiwania na odpowiedź. Dlatego do kolejnych etapów, już nastawionych na skracanie opóźnienia, wybrano wyłącznie rodzinę Gemma.

Puła modeli w etapie kwantyzacji nie pokrywa się więc z pulą Optuny. Z pierwszej fazy badań pozostał zwycięzca jakościowy. Do testu precyzji wag dołączono natomiast gemma4-e2b, model z tej samej rodziny, który w dotychczasowych próbach reagował na

stosowane metody optymalizacji oraz polski model SLM bielik-4.5b, wcześniej zdecydowanie zbyt wolny, by sprawdzić, czy obniżenie precyzji wag pozwoli mu wejść do konfiguracji produkcyjnej.

Samo obniżenie precyzji wag początkowo nie spowodowało oczekiwanego skrócenia czasu generacji. Metryka E2E dla wariantów Q4 i Q8 wykazywała wciąż niezadowalające wyniki (ponad 20s opóźnienia), ponieważ pomiary dominował pełny prefill przy *cold cache*. Dopiero po przejściu wywołaniu mechanizmu *warm cache*, gdy model i KV-cache pozostają w RAM urządzenia brzegowego, a prefill ogranicza się do nowego sufiksu, wpływ kwantyzacji stał się widoczny. Ilustruje to poniższa tabela 5.14.

Tabela 5.14. Porównanie czasów E2E dla cold i warm cache.

Model	Wariant	$e^{\hat{\mu}}$ cold [s]	$e^{\hat{\mu}}$ warm [s]
gemma3-4b	Q8_0	71,2	12,3
gemma3-4b	Q4_K_M	37,9	9,0
gemma3-4b	Q4_0	38,5	8,1
gemma4-e2b	Q4_K_M	26,7	6,3
bielik-4.5b	Q4_K_M	86,6	41,7

Wpływ kwantyzacji na całkowity czas odpowiedzi zależy od trybu pracy (cold lub warm cache) pełną analizę, matematyczne modelowanie oraz dekompozycję czasu generacji E2E na składowe przedstawiono w sekcji 5.3. Przy trybie *cold cache* mediana LN dla gemma3-4b pozostaje w okolicach kilkadziesiątu sekund (ok. 38 s przy Q4, ok. 71 s przy Q8_0), samo obniżenie precyzji wag nie daje zatem zadowalających wyników dla asystenta głosowego. Po przejściu na tryb *warm cache* ten sam model osiąga czas E2E ok. 9 s (Q4_K_M) i ok. 8 s (Q4_0). Nowszy model z rodziny gemma4-e2b jest jeszcze szybszy (ok. 6 s), lecz agresywna kwantyzacja Q2_K destabilizuje routing w architekturze MoE. bielik-4.5b nawet po Q4_K_M zostaje przy bardzo wysokim wyniku ok. 42 s, przez co jego zastosowanie produkcyjne zostało wykluczone.

Skoro zysk z *warm cache* możliwy jest do uzyskania dopiero przy zastosowaniu stałego początku sekwencji, prompt systemowy złożono z dwóch części: prefiks- części stałej (persona i reguły) oraz sufiks- części zmiennej (fakty, Findings Nauczyciela, historia, bieżące pytanie). Dokładne omówienie tej struktury przedstawiono w sekcji 5.4.

5.5.2. Rekomendacja produkcyjna

Na podstawie wyników omówionych w sekcji powyżej wybrano konfigurację produkcyjną modelu Ucznia, która została zaimplementowana na urządzeniu brzegowym Raspberry Pi 5. Zestawia ona zwycięzcę jakościowego fazy Optuny z wariantem kwantyzacji, który przy *warm cache* daje czytelne przyspieszenie generacji bez utraty związku, wraz z strukturą promptu umożliwiającym ponowne użycie KV-cache.

Tabela 5.15. Rekomendowana konfiguracja produkcyjna Ucznia.

Parametr	Wartość
Model	gemma3:4b
Kwantyzacja	Q4_K_M
temperature	0,20
num_ctx	2048
num_predict	128
top_p	0,90
repeat_penalty	1,05
keep_alive	30 min
Prompt	prefiks stały + sufiks zmienny (tabela 5.12)
TTFT (<i>warm</i> , LN)	≈ 2,4 s
Generacja (<i>warm</i> , LN)	≈ 6,4 s
E2E (<i>warm</i> , LN)	≈ 9 s

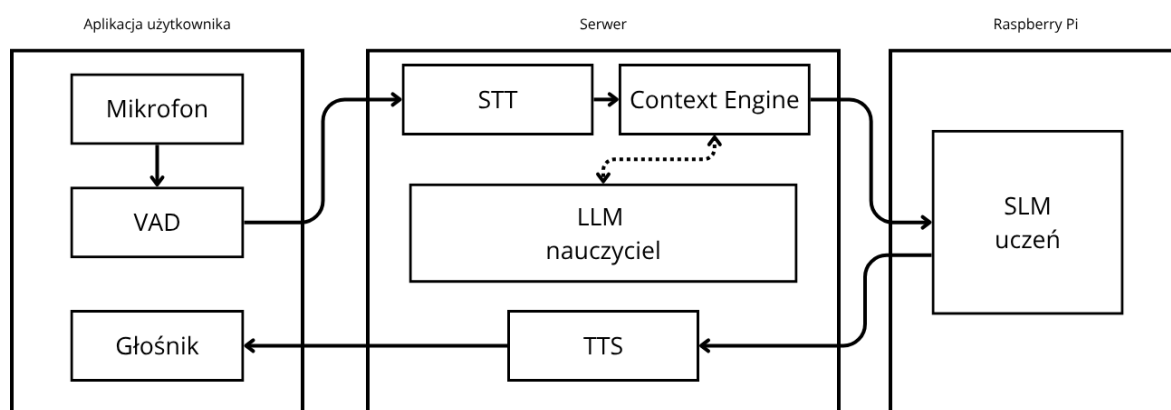
Układ promptu jest tożsamy z konstrukcją opisaną w tabeli 5.12: stały prefiks systemowy pozostaje niezmienny między zapytaniami, a elementy zmienne trafiają wyłącznie do sufiksu, co umożliwi aktywowanie trybu *prefix caching*.

Wybrana konfiguracja łączy najniższy spośród badanych modeli wskaźnik halucynacji z opóźnieniem akceptowalnym dla asystenta głosowego na urządzeniu brzegowym. Dalsze skracanie E2E przez agresywniejszą kwantyzację lub strojenie hiperparametrów ciągnie za sobą pogorszenie jakości odpowiedzi.

6. Realizacja prototypu

6.1. Architektura prototypu

Zaimplementowany finalny system asystenta głosowego Wilga odpowiada oparty jest na koncepcji z rozdziału 3.3, składają się na nią: aplikacja użytkownika, serwer orkiestrujący oraz węzeł brzegowy Raspberry Pi 5. Schemat wdrożenia przedstawiono na rysunku 6.1, zastosowano ten sam podział na trzy środowiska co na rysunku schematu testowego 3.1, w przypadku prezentowanej wdrożeniowej wersji asystenta przedstawiono architekturę wraz z konkretnymi procesami i kierunkami przepływu danych.



Rysunek 6.1. Architektura prototypu Wilga.

Architektura produkcyjna jest pokrewna, lecz nie tożsama z układem benchmarkowym opisanym w sekcji 5.1 (rysunek 5.1). Wspólnym elementem pozostaje główny aktor: węzeł brzegowy, na który składają się serwer Ollama oraz mały model językowy (SLM) Uczeń na Raspberry Pi 5. Względem architektury testowej największe zmiany poczyniono w warstwie serwerowej. W pomiarach orkiestratorem był skrypt testowy oraz złoty zbiór pytań (*golden set*); w prototypie tę rolę przejmuje pełny łańcuch głosowy (aktywowany przez użytkownika) oraz Context Engine (działający asynchronicznie). Konfiguracja modelu Ucznia pochodzi z rekomendacji badawczej (tabela 5.15), wybrano *gemma3:4b*, kwantyzacja *Q4_K_M*, *keep_alive=30m* oraz stały prefiks promptu pod *prefix caching*.

Przepływ danych wywołany zapytaniem użytkownika przebiega wzdłuż strzałek na rysunku 6.1. Mikrofon przez przeglądarkę rejestruje sygnał audio. Detekcja aktywności mowy (ang. *Voice Activity Detection*, VAD) w aplikacji kończy nagranie po chwili ciszy; start następuje po wywołaniu frazy „Hej Wilguś” albo po kliknięciu postaci na ekranie. Segment audio trafia na serwer, gdzie transkrypcja (ang. *Speech-to-Text*, STT) zamienia mowę na tekst. Context Engine dokłada do pytania stały prefiks reguł, dane z czujników oraz bieżący kontekst wygenerowany przez Nauczyciela. Tak złożone żądanie trafia na urządzenie brzegowe Raspberry Pi, do modelu SLM Ucznia. Wygenerowana odpowiedź wraca na serwer, synteza mowy (ang. *Text-to-Speech*, TTS) produkuje audio, które jest transportowane do aplikacji użytkownika gdzie głośnik odtwarza odpowiedź.

Context Engine nie stoi na ścieżce krytycznej zapytania. Pracuje w dwóch cyklach. Cykl asynchroniczny, sterowany interwałem `CONTEXT_REFRESH_INTERVAL` (domyślnie 300 s), w tle przesuwając kursor po historycznych odczytach IoT (rekordy bazy danych), odpytuje Nauczyciela i buduje kontekst na bieżąco. Całość odbywa się przy pomocy skryptów w języku Python w sposób cykliczny i asynchroniczny, niezależnie od użytkownika. Cykl synchroniczny skupia się jedynie na doklejeniu aktualnego kontekstu do pytania użytkownika oraz przed przesłaniem zapytania do modelu Ucznia. Czas odpowiedzi asystenta nie zależy więc od czasu generacji Nauczyciela. Linia przerywana na schemacie oznacza właśnie tę asynchroniczną, dwukierunkową współpracę silnika kontekstu modelem LLM.

Podział maszyn jest zgodny z założeniem o odciążeniu urządzenia brzegowego. Komputer Mac (serwer) wykonuje transkrypcję, syntezę mowy, budowanie kontekstu i inferencję Nauczyciela. Raspberry Pi udostępnia wyłącznie interfejs Ollama API, tak aby całe zasoby były przeznaczone dla modelu SLM.

6.2. Elementy systemu i wykorzystane technologie

W poniższej sekcji skupiono się na kolejnych elementach systemu, ich roli oraz wykorzystanych technologiach.

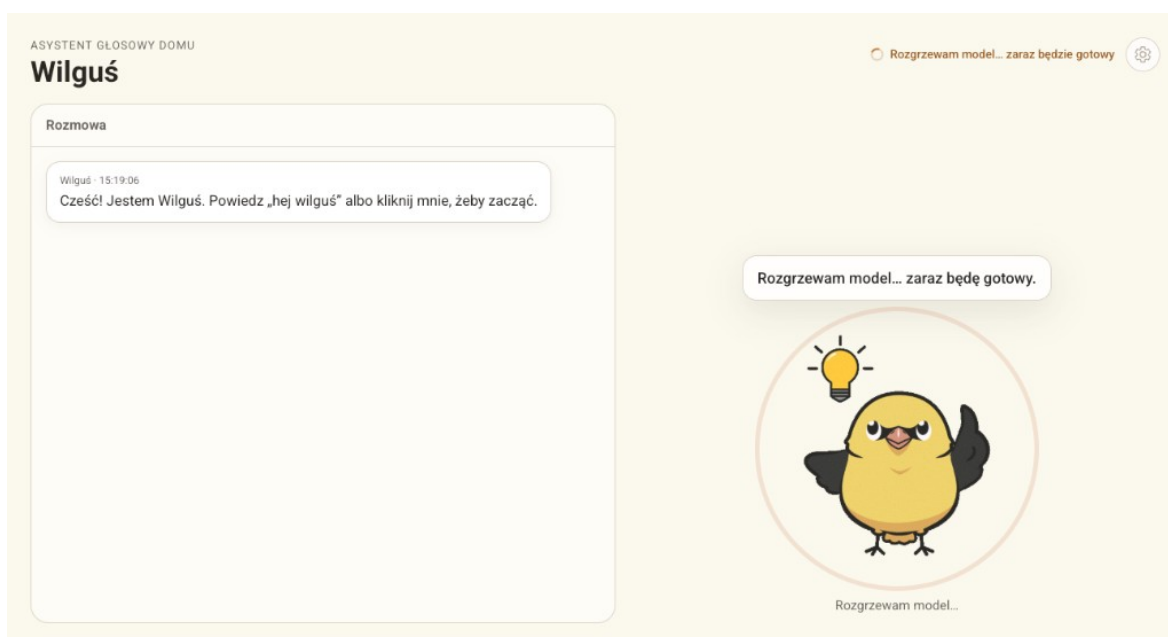
6.2.1. Aplikacja użytkownika

Interfejs użytkownika zrealizowano w React z wykorzystaniem bundlera Vite, za część stylistyczną odpowiada Tailwind CSS. Aplikacja nie transkrybuje mowy ani nie syntetyzuje odpowiedzi. Jej zadania ograniczają się do: nagrania audio, wyświetlania historii rozmowy oraz wizualnych zmian stanów, takich jak animacja maskotki Wilguś. Wiele wyborów technologicznych dotyczących tej warstwy wynikało z dziedziczenia koncepcji architektonicznych poprzedniej aplikacji Wilga MiLL, co opisano w rozdziale 3.

System obsługuje dwa tryby aktywacji. W trybie głosowym przeglądarka nasłuchuje frazy kluczowej „Hej Wilguś” przez Web Speech API (Chrome/Edge). Rozpoznawane są także warianty transkrypcji (obsługa błędów) (hej wilga, hej wilgo, hey wilgus i podobne); po wykryciu frazy kluczowej frontend obcina frazę z początku wypowiedzi, żeby asystent nie traktował jej jako pytania, jest ona jedynie wywołaniem systemu. W trybie ręcznym nagranie startowane jest poprzez kliknięciem postaci Wilgusia na ekranie. W obu przypadkach ten sam odcinek jest nagrywany i przesyłany do systemu.

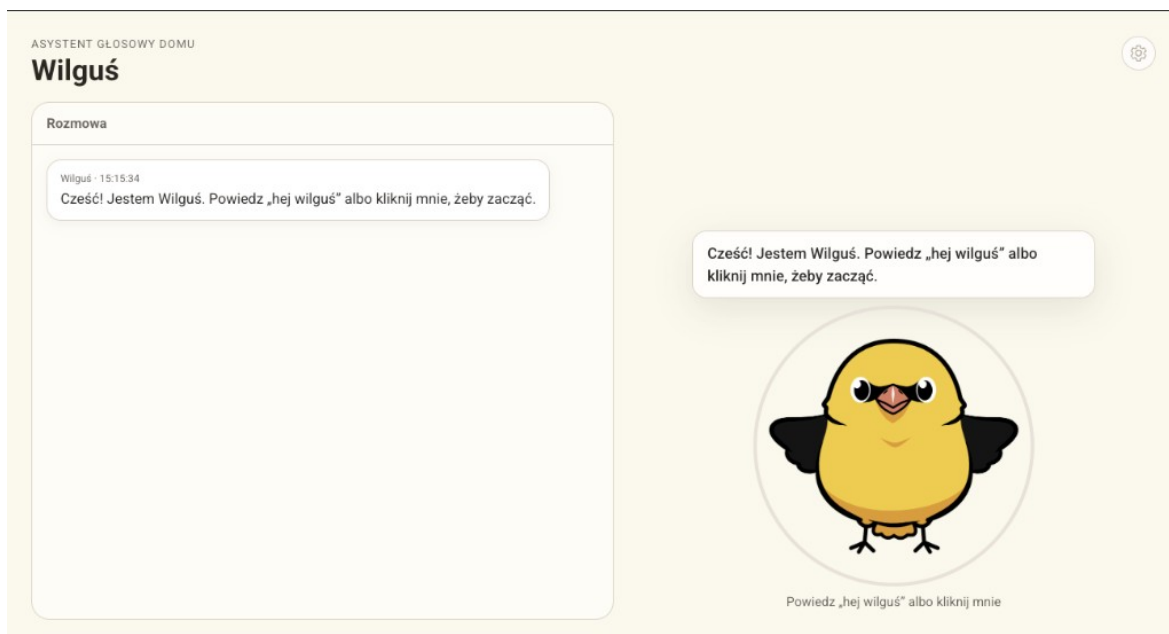
Mechanizm znalezienia momentu, w którym nagrywanie zostaje wstrzymane oparto o prosty detektor poziomu głośności badany przez mikrofon urządzenia końcowego, a nie o model Silero omawiany w przeglądzie literatury. Utrzymanie się poniżej wyznaczonego progu przez 1,5 s kończy nagranie; dolny i górny limit trwania to 2,5 s oraz 20 s. Rozwiązanie jest lepsze niż neuronowy VAD i w testach wykazało, że dobrze sprawdza się do naturalnego ucięcia wypowiedzi bez instalowania dodatkowego modelu na serwerze. Wyznaczenie końca nagrania nie jest zatem adaptacyjne. Poziom, który uznano za ciszę jest stałą w kodzie aplikacji. W domkach w Wildzie, które znajdują się w lesie panują dość dobre warunki akustyczne. Niemniej dobrym kierunkiem rozwoju byłoby adaptacyjne ustawienie progu lub filtrowanie częstotliwości zbieranych przez mikrofon.

Układ interfejsu użytkownika jest dwukolumnowy. Po lewej stronie znajduje się karta *Rozmowa* gdzie wyświetlany jest czat, po prawej widnieje postać Wilgusia (animowana w zależności od statusu). Aplikacja na starcie po uruchomieniu, przechodzi przez fazę przygotowawczą. Zanim użytkownik może zadać pierwsze pytanie, model Uczeń musi być przygotowany, fazę tę nazwano rozgrzewaniem. Faza ta ma zapobiegać wydłużonej odpowiedzi na pierwsze zapytanie, które trafiłoby w *cold cache* i czas do pierwszego tokenu (TTFT) sięgałby rzędu dziesiątek sekund, tak jak wykazano to w sekcji 5.3. Pętla `StudentKeepWarm` razem z `keep_alive` ustawionym na 30 min powodują, że zarówno wagi modelu jak i KV-cache stałego prefiksu ciągle znajduje się w RAM urządzenia brzegowego co zapobiega jego ładowania do pamięci podręcznej w momencie pojawienia się zapytania użytkownika. Frontend odpytuje endpointu `GET /api/health`, który w czasie rozgrzewania modelu zwraca status `warming` co blokuje interakcję użytkownika. Widok prezentowany użytkownikowi podczas fazy przygotowawczej przedstawiono na rysunku 6.2.



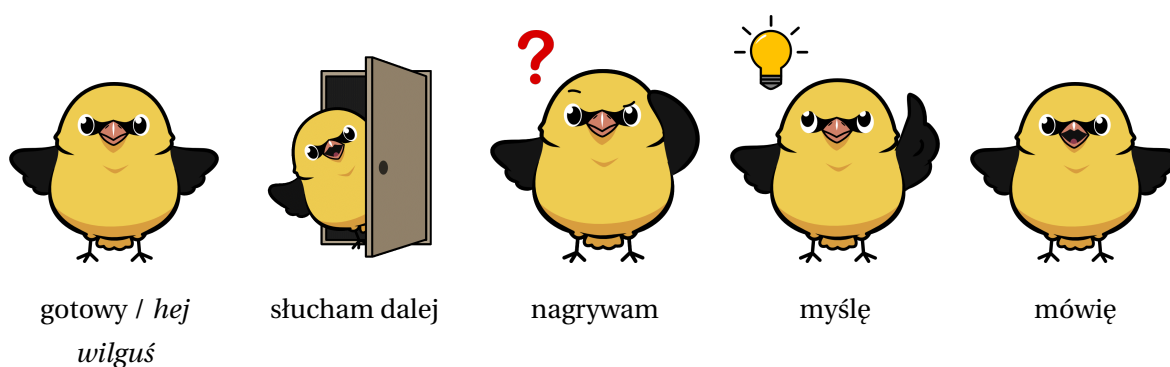
Rysunek 6.2. Faza rozgrzewania modelu Ucznia.

Po zakończeniu procesu rozgrzewania modelu Ucznia użytkownikowi prezentowany jest delikatnie inny widok- ekran startowy (rysunek 6.3): powitanie, subtelne instrukcje jak rozpocząć rozmowę (frazą startową lub kliknięcie Wilgusia). Dodatkowo aby zachęcić do interakcji postać Wilgusi pulsuje pokazując swoją gotowość do interakcji. Widok startowy nadal trzyma omówiony powyżej dwukolumnowy układ.



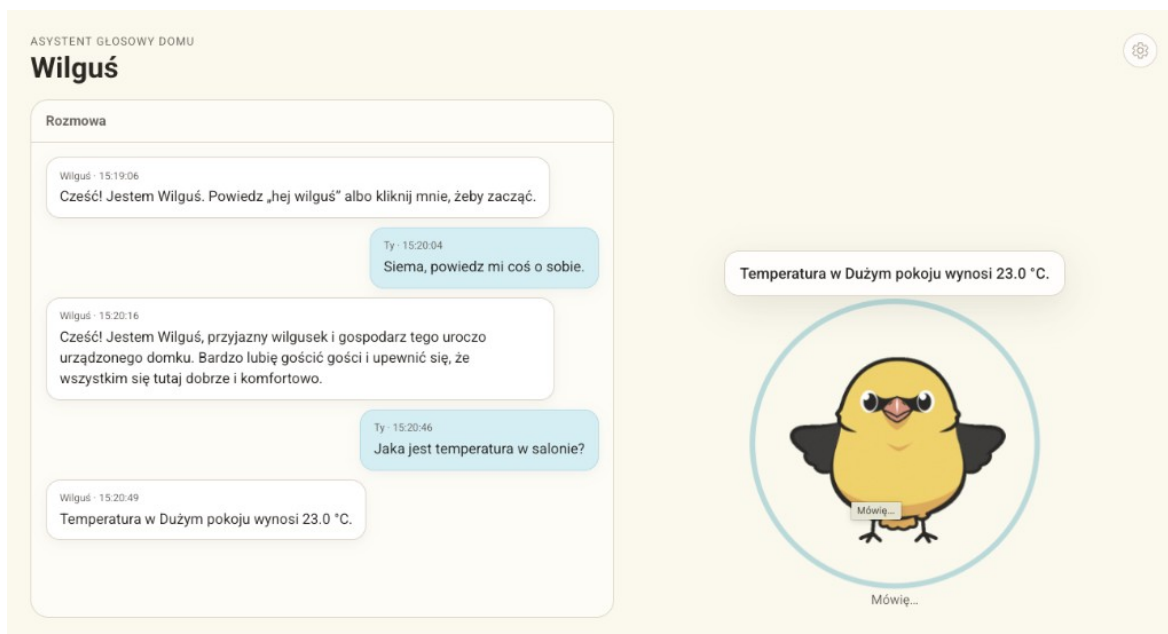
Rysunek 6.3. Ekran startowy po rozgrzaniu modelu.

Wilguś widoczny w prawej kolumnie prezentowanych widoków nie jest statyczną ilustracją. Tablica `WILGUS_BY_STATUS` przypisuje fazom czy statusom aplikacji osobną grafikę dynamiczną i podpis, dzięki czemu postać sygnalizuje, co system właśnie robi. Grafiki zapisane w formacie GIF wprowadzają element ruchu, który podnosi zaangażowanie użytkownika. Klatki faz używanych w prototypie zestawiono na rysunku 6.4. Postać Wilgusia została zaprojektowana przez Julię Nazarską, studentkę wydziału Architektury Politechniki Warszawskiej.



Rysunek 6.4. Stany Wilgusia w aplikacji użytkownika.

Podczas konwersacji lewa kolumna wypełnia się wiadomościami użytkownika i odpowiedziami asystenta. Całość ma przypominać dobrze znany interfejs chatu, przez co korzystanie z aplikacji jest intuicyjne. Na rysunku 6.5 przedstawiono przykładowy widok w trakcie prowadzenia rozmowy.



Rysunek 6.5. Widok aplikacji w trakcie prowadzenia rozmowy.

6.2.2. Transkrypcja mowy (STT)

Za transkrypcję mowy, czyli element procesu nazwany STT na schemacie architektonicznym (rysunek 6.1) odpowiada blok roboczo nazwany Whisper Backend. Jest to usługa stworzona w oparciu o bibliotekę języka Python FastAPI oraz otwartoźródłowego modelu Whisper. Sercem modułu jest transkrypcja, którą realizuje biblioteka `faster-whisper` [23]. W prezentowanym rozwiązaniu zdecydowano się implementować wariant `small`, oczywiście w konfiguracji wskazano język polski (`pl`). Filtr VAD po stronie Whisper jest wyłączony (`WHISPER_VAD_FILTER=false`), ponieważ w systemie istnieje własny detektor aktywności, widoczny na schemacie architektonicznym pod nazwą VAD.

Nagranie zebrane przez aplikację użytkownika zostaje wysłane do Whisper Backendu poprzez żądanie `HTTP POST /api/transcribe`, pole `file` zawiera plik nagrania (mechanizm znany z formularzy aplikacji webowych `multipart/form-data`). Odpowiedź zwracana jest w formacie JSON, jej przykład przedstawiono poniżej:

```
{"text": "jaka jest temperatura w salonie",  
  "language": "pl",  
  "duration_seconds": 2.4}
```

Moduł STT został zabezpieczony przez błędami. Serwis odrzuca nagrania większe niż 25 MB oraz niemal puste pliki (poniżej ok. 1,5 kB), brak reakcji na szum.

6.2.3. Context Engine

Kolejnym elementem systemu jest Context Engine, zestaw skryptów Python również wystawiających styk poprzez bibliotekę FastAPI. Ze względu na brak danych spływających z Wilgi na żywo w trakcie prac nad opisywaną pracą magisterską, zdecydowano zaprojektować imitację. Serwis czyta plik CSV będących eksportem z realnych rekordów zebranych

z bazy danych w czasie gdy system funkcjonował poprawnie (okres 31.10.2025–27.01.2026). Każdy cykl odświeżania przesuwa moment w czasie o jeden wiersz, czyli 5 minut, ponieważ co tyle były agregowane dane w bazie danych. Do promptu nie idzie pełna szerokość tabeli, skrypt pomija metryki mówiące o stanie technicznym np. poziomowi baterii czy jakości łącza nieistotne z punktu widzenia użytkownika.

Fakty dla Ucznia składane są przez skrypt deterministycznie: z wiersza CSV powstaje tak zwany *snapshot* z etykietami oznaczającymi pomieszczenia (łazienka, duży pokój, mały pokój, korytarz). Ten sam *snapshot* dostarczony jest do Nauczyciela aby on przygotował analizę anomalii. Działanie obu wątków jest niezależne to znaczy, jeśli nauczyciel nie odpowie context engine będzie dostarczał snapshot do Ucznia bez względu na niedostępność większego modelu.

Kontekst ma dwa bufory, w których przechowany jest snapshot kontekstu: *Published* przypięty do bieżącej rozmowy, *staging* odświeża się w tle, zapewniając że snapshot będzie możliwie aktualny w momencie rozpoczęcia nowej rozmowy. Wywołanie nowej sesji (POST /api/session/new po stronie Whisper Backendu) powoduje wywołanie metody POST /api/context/advance i przestawia publikowany snapshot na nowszy. Niezmienny oczywiście pozostaje prefiks, snapshot jest częścią sufiksu, dzięki temu KV-cache na Pi pozostaje ważny między turami.

Whisper Backend pobiera bieżący kontekst przez endpoint GET /api/context. Przykładowa odpowiedź serwera:

```
{
  "status": "ready",
  "source_timestamp": "2025-11-01T12:00:00Z",
  "static_prefix": "Jesteś Wilguś -- ...",
  "dynamic_context": "Stan mieszkania na ... UTC: ...",
  "insight": "Okno w korytarzu otwarte.",
  "system_prompt": "Jesteś Wilguś -- ...\n\nStan mieszkania...".
}
```

Pole *static_prefix* (niezmienna część promptu) składa się z osoby, osoby Wilguś oraz reguł, których powinien przestrzegać. Pole *dynamic_context* to surowe dane maszynowo opakowane w etykiety przez Context Engine. Ostatnią składową jest sekcja Findings, zawiera ona analizę anomalii wykonaną przez Nauczyciela. Pole *system_prompt* jest złożeniem obu części, stałej oraz zmiennej. De facto to z niego korzystać będzie Uczeń dalej w łańcuchu przetwarzania.

6.2.4. LLM Nauczyciel

Model LLM zwany Nauczycielem ze względu na jego rolę w systemie działa lokalnie na komputerze MacBook. Dzięki aplikacji Ollama uruchamianie modeli i udostępnianie API w sieci lokalnej jest bardzo proste. Jako model Nauczyciela wybrano Qwen2.5 w konfiguracji 4b, nie poczyniono żadnych prac optymalizacyjnych. Ten węzeł jest jedynie symulacją zewnętrznego systemu analitycznego. Zadanie jest wąskie: nie streszczać stanu całego domu

wczasowego, wskazać odchylenia od normy (skrajne temperatury, otwarte okna, nietypowa obecność, sprzeczne odczyty). Gdy nic nie zostanie sklasyfikowane jako anomalia, model ma zwrócić dokładnie jedną linię BRAK_ANOMALII, serwis (Context Engine) traktuje to jako sygnał do pominięcia pola Findings podczas składania System Promptu Ucznia.

Żądanie przyjmowane przez API Ollamy ma postać zwaną *OpenAI-compatible*, schemat wzorowany na API zaproponowanym przez firmę OpenAI, przykładowa wiadomość to:

```
{
  "model": "qwen2.5",
  "messages": [
    {"role": "system", "content": "Jesteś analitykiem IoT smart-home..."},
    {"role": "user", "content": "Snapshot czujników z ..."}
  ],
  "stream": false,
  "keep_alive": "30m"
}
```

6.2.5. SLM Uczeń

Modelem ucznia jest gemma3:4b uruchomiony na urządzeniu brzegowym Raspberry Pi (Ollama udostępnia API w sieci lokalnej, tak samo jak Nauczyciel), Model SLM uruchomiony jest w konfiguracji przedstawioną w tabeli 5.15. Zapytania modelu LLM Ucznia wykonywane są przez skrypty Python (Context Engine), przykładowa wiadomość ma format:

```
{
  "message": "Jaka jest temperatura w salonie?",
  "history": [
    {"role": "user", "content": "Siema, powiedz mi coś o sobie"},
    {"role": "assistant", "content": "Jestem Wilguś, gospodarz domku..."}
  ]
}
```

Context Engine wzbogacany jest o poprzednie pary pytanie-odpowiedź, maksymalnie 3 ostatnie pary. Payload budowany przez Context Engine dba o to by prefiks systemowy był bitowo stały, co opisano w podrozdziale powyżej. Ten zabieg w połączeniu z konfiguracją Ollama (włączenie stream: true oraz keep_alive=true) pozwala strumieniować tokeny i utrzymać ciepły cache. Odpowiedź zwracana przez model podzielona jest na chunki, czyli kawałki wiadomości, dzięki takiemu podziałowi możliwe jest przetwarzanie na dźwięk zdania po zdaniu, co zmniejsza czas TTFT. Przykładowa odpowiedź modelu w formacie NDJSON:

```
{"type": "chunk", "content": "Temperatura w Dużym pokoju"}
{"type": "chunk", "content": " wynosi 23,0 °C."}
{"type": "done"}
```

Węzeł Ucznia posiada podstawową obsługę błędów. Zdarzenie error zwracane jest przez system gdy przekroczone zostanie 120 s opóźnienia albo przy błędzie sieciowym (brak połączenia z Uczniem). Gdy Context Service nie odpowiada asystent głosowy odpowiada wyłącznie bazując na stałym prefiksie systemowym. Instrukcje przekazują Uczniowi zakaz zmyślania informacji, odpowiada on jedynie na pytania dotyczące jego tożsamości i zasad zachowania.

6.2.6. Synteza mowy (TTS)

Moduł TTS przyjmuje dane w postaci chunk'ów zwracanych przez Ucznia. Zdaniowe fragmenty są kolejgowane i przetwarzane w sposób sekwencyjny przez bibliotekę `edge-tts`. Odpowiedź generowana jest w formacie `audio/mpeg`. Biblioteka oferuje dwa głosy w języku polskim, męski oraz żeński. Zdecydowano się skorzystać z głosu żeńskiego `pl-PL-ZofiaNeural`, ze względu na charakter postaci Wilgusia. Idealny byłby głos dziecięcy, jednak takie zakazane są w Unii Europejskiej **EUAIAct2024**. Pakiet `edge-tts` jest darmowy i otwartoźródłowy, jednak synteza mowy realizowana jest przez chmurowy serwis (Microsoft Azure). Jest to niekorzystne projektowo, ponieważ wymaga dostępu do sieci internetowej. Wada ta powoduje, że rozsądnym kierunkiem rozwoju byłoby zastąpienie tej warstwy lokalnym modelem TTS, niezależnym od internetu.

6.2.7. Konfiguracja

Pełną konfigurację systemu przedstawiono w załączniku A. (do dopisania)

6.3. Testy rozwiązania

7. Podsumowanie

Bibliografia

- [1] H. Renney, F. Trad, M. Mattarock i Z. Wood, “Cloud to Edge: Benchmarking LLM Inference On Hardware-Accelerated Single-Board Computers”, *arXiv preprint arXiv:2604.24785*, 2026, Carnegie Mellon University; University of Exeter. URL: <https://arxiv.org/abs/2604.24785>
- [2] T. Nguyen i in., “An Evaluation of LLMs Inference on Popular Single-board Computers”, *arXiv preprint arXiv:2511.07425*, 2025, University of Technology Sydney. URL: <https://arxiv.org/abs/2511.07425>
- [3] P. Tummalapalli, S. Arayakandy, R. Pal i K. Kundan, “LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load”, *arXiv preprint arXiv:2603.23640*, 2026. URL: <https://arxiv.org/abs/2603.23640>
- [4] A. Vaswani i in., “Attention is all you need”, *Advances in neural information processing systems*, t. 30, 2017, Seminal paper on Transformer architecture.
- [5] A. Dubey, A. Jauhri, A. Pandey, A. Keshwam, A. Al-Dahle i in., “The Llama 3 Herd of Models”, *Meta Publications*, 2024, Meta AI Technical Report. URL: <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- [6] M. Abdin, S. A. Jacobs, A. A. Awan, J. Bi i in., “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone”, *arXiv preprint arXiv:2404.14219*, 2024, Microsoft Research. URL: <https://arxiv.org/abs/2404.14219>
- [7] G. Team i G. DeepMind, “Gemma 2: Improving Open Language Models at a Practical Size”, *Semantic Scholar*, 2024, Google DeepMind Technical Report. URL: <https://www.semanticscholar.org/paper/Gemma-2%3A-Improving-Open-Language-Models-at-a-Size-Riviere-Pathak>
- [8] SpeakLeash Team, *Bielik-7B-v2: Polish Language Model*, <https://huggingface.co/speakleash/Bielik-7B-v2>, Software, Accessed: 2024-10-15, 2024.
- [9] S. Kim i in., “SqueezeLLM: Dense-and-Sparse Quantization”, w: *International Conference on Machine Learning*, UC Berkeley, 2023. URL: <https://arxiv.org/abs/2306.07629>
- [10] E. Frantar, S. Ashkboos, T. Hoefler i D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers”, w: *International Conference on Learning Representations*, ETH Zurich, 2023. URL: <https://arxiv.org/abs/2210.17323>
- [11] U. Kurt, “Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct”, *arXiv preprint arXiv:2601.14277*, 2026. URL: <https://arxiv.org/abs/2601.14277>
- [12] G. Hinton, O. Vinyals i J. Dean, “Distilling the knowledge in a neural network”, *arXiv preprint arXiv:1503.02531*, 2015, Foundational paper on Knowledge Distillation. URL: <https://arxiv.org/abs/1503.02531>
- [13] L. Chen, M. Zaharia i J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance”, *arXiv preprint arXiv:2305.05176*, 2023, Stanford University. URL: <https://arxiv.org/abs/2305.05176>

- [14] Y. Leviathan, M. Kalman i Y. Matias, “Fast Inference from Transformers via Speculative Decoding”, w: *Proceedings of the 40th International Conference on Machine Learning*, Google, PMLR, 2023, s. 19 274–19 286. URL: <https://arxiv.org/abs/2211.17192>
- [15] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu i in., “The Prompt Report: A Systematic Survey of Prompting Techniques”, *arXiv preprint arXiv:2406.06608*, 2024, University of Maryland. URL: <https://arxiv.org/abs/2406.06608>
- [16] T. B. Brown i in., “Language Models are Few-Shot Learners”, w: *Advances in Neural Information Processing Systems*, OpenAI, t. 33, 2020, s. 1877–1901. URL: <https://arxiv.org/abs/2005.14165>
- [17] J. Wei i in., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, w: *Advances in Neural Information Processing Systems*, Google Research, t. 35, 2022, s. 24 824–24 837. URL: <https://arxiv.org/abs/2201.11903>
- [18] P. Lewis i in., “Retrieval-augmented generation for knowledge-intensive nlp tasks”, *Advances in Neural Information Processing Systems*, t. 33, s. 9459–9474, 2020, Key paper introducing RAG.
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta i M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework”, w: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Preferred Networks, 2019, s. 2623–2631. URL: <https://arxiv.org/abs/1907.10902>
- [20] E. J. Hu i in., “LoRA: Low-Rank Adaptation of Large Language Models”, w: *International Conference on Learning Representations*, Microsoft Research, 2022. URL: <https://arxiv.org/abs/2106.09685>
- [21] T. Detrmers, A. Pagnoni, A. Holtzman i L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms”, *Advances in Neural Information Processing Systems*, t. 36, 2024, Techniques for running LLMs on consumer hardware. URL: <https://arxiv.org/abs/2305.14314>
- [22] Silero Team, *Silero VAD: pre-trained enterprise-grade Voice Activity Detector*, <https://github.com/snakers4/silero-vad>, MIT License, 2021.
- [23] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey i I. Sutskever, “Robust speech recognition via large-scale weak supervision”, w: *International Conference on Machine Learning*, OpenAI Whisper Technical Report, PMLR, 2023, s. 28 492–28 518.
- [24] ElevenLabs, *ElevenLabs: AI Voice Generator and Text-to-Speech Platform*, <https://elevenlabs.io>, Commercial TTS / voice AI platform, 2024.
- [25] J. Kim, J. Kong i J. Son, “Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech”, w: *International Conference on Machine Learning*, VITS: State-of-the-art Model for TTS, PMLR, 2021, s. 5530–5540.
- [26] rany2, *edge-tts: Use Microsoft Edge’s online text-to-speech service from Python*, <https://github.com/rany2/edge-tts>, Python wrapper for Microsoft Edge neural TTS voices, 2021.

-
- [27] Apple, *Explore the new system architecture of Apple silicon Macs*, WWDC20 Session 10686, Oficjalny opis Unified Memory Architecture w Apple Silicon, 2020. URL: <https://developer.apple.com/videos/play/wwdc2020/10686/>
 - [28] Apple, *Metal Compute on MacBook Pro*, Apple Tech Talks, Dostęp GPU do pamięci systemowej w Unified Memory Architecture, 2021. URL: <https://developer.apple.com/videos/play/tech-talks/10580/>
 - [29] S. Es, J. James, L. Espinosa-Anke i S. Schockaert, “Ragas: Automated Evaluation of Retrieval Augmented Generation”, w: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, RAGAS: faithfulness, answer relevance, 2024, s. 150–158. URL: <https://aclanthology.org/2024.eacl-demo.16/>
 - [30] S. M. Ghafari i in., “ConCISE: A Reference-Free Conciseness Evaluation Metric for LLM-Generated Answers”, *arXiv preprint arXiv:2511.16846*, 2025, Metryka concise-ness odpowiedzi LLM. URL: <https://arxiv.org/abs/2511.16846>
 - [31] A. B. M. A. Rahman, S. Anwar, M. Usman i A. Mian, “DefAn: Definitive Answer Dataset for LLMs Hallucination Evaluation”, *arXiv preprint arXiv:2406.09155*, 2024, Benchmark halucynacji i spójności odpowiedzi. URL: <https://arxiv.org/abs/2406.09155>
 - [32] R. Pope i in., “Efficiently Scaling Transformer Inference”, w: *Proceedings of Machine Learning and Systems*, MLSys 2023; dekompozycja prefill vs decode, t. 5, 2023. URL: <https://arxiv.org/abs/2211.05102>
 - [33] W. Kwon i in., “Efficient Memory Management for Large Language Model Serving with PagedAttention”, w: *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP 2023; KV-cache i PagedAttention, 2023, s. 611–626. URL: <https://arxiv.org/abs/2309.06180>
 - [34] Ollama, *Ollama API documentation*, <https://github.com/ollama/ollama/blob/main/docs/api.md>, Parametr keep_alive; domyślnie 5 minut, 2024.
 - [35] Raspberry Pi Ltd, *Broadcom BCM2712*, Raspberry Pi Documentation, 32-bit LPDDR4X, przepustowość do 17 GB/s, 2023. URL: <https://github.com/raspberrypi/documentation/blob/develop/documentation/asciidoc/computers/processors/bcm2712.adoc>

Wykaz symboli i skrótów

AI – sztuczna inteligencja (ang. *Artificial Intelligence*)

API – interfejs programowania aplikacji (ang. *Application Programming Interface*)

ARM – architektura procesorów o zredukowanej liście rozkazów (ang. *Advanced RISC Machine*)

CPU – centralna jednostka obliczeniowa (ang. *Central Processing Unit*)

CSV – plik wartości rozdzielonych przecinkami (ang. *Comma-Separated Values*)

E2E – czas odpowiedzi od zapytania do wygenerowania ostatniego tokenu (ang. *End-to-End*)

EiTI – Wydział Elektroniki i Technik Informacyjnych

GGUF – format zapisu wag modelu językowego (ang. *GPT-Generated Unified Format*)

GPTQ – metoda kwantyzacji po trenowaniu wykorzystująca przybliżenie drugiego rzędu (ang. *Generative Pre-trained Transformer Quantization*)

GPU – układ graficzny (ang. *Graphics Processing Unit*)

HTTP – protokół przesyłania hipertekstu (ang. *Hypertext Transfer Protocol*)

IoT – Internet Rzeczy (ang. *Internet of Things*)

JSON – format zapisu i wymiany danych (ang. *JavaScript Object Notation*)

KV – pary klucz–wartość (ang. *key–value*); w pracy w kontekście mechanizmu KV-cache

LLM – duży model językowy (ang. *Large Language Model*)

LN – rozkład log-normalny (ang. *Log-Normal Distribution*)

LoRA – adaptacja niskorzędowa (ang. *Low-Rank Adaptation*)

MoE – mieszanina ekspertów (ang. *Mixture-of-Experts*)

NDJSON – format JSON rozdzielany znakami nowej linii (ang. *Newline Delimited JSON*)

NLP – przetwarzanie języka naturalnego (ang. *Natural Language Processing*)

PTQ – kwantyzacja po trenowaniu (ang. *Post-Training Quantization*)

PW – Politechnika Warszawska

QLoRA – adaptacja LoRA z kwantyzacją wag bazowych (ang. *Quantized Low-Rank Adaptation*)

RAG – generacja wspomagana wyszukiwaniem (ang. *Retrieval-Augmented Generation*)

RAM – pamięć operacyjna (ang. *Random Access Memory*)

RPi – Raspberry Pi

SLM – mały model językowy (ang. *Small Language Model*)

SoC – układ scalony integrujący wszystkie elementy systemu (ang. *System on a Chip*)

STT – zamiana mowy na tekst (ang. *Speech-to-Text*)

TPE – estymator Parzena o strukturze drzewiastej (ang. *Tree-structured Parzen Estimator*)

TTFT – czas do wygenerowania pierwszego tokenu (ang. *Time To First Token*)

TTS – synteza mowy (ang. *Text-to-Speech*)

VAD – detekcja aktywności głosowej (ang. *Voice Activity Detection*)

VRAM – pamięć karty graficznej (ang. *Video RAM*)

Spis rysunków

3.1	Architektura systemu.	20
3.2	Wilguś w aplikacji użytkownika.	21
5.1	Schemat systemu benchmarkowego.	30
5.2	Porównanie R^2 modeli regresji dla composite score.	34
5.3	Zależność <i>composite score</i> od wskaźnika halucynacji.	35
5.4	Rozpiętość <i>nonhall_score</i> i <i>composite score</i>	36
5.5	Wskaźnik halucynacji per przypadek testowy.	37
5.6	Histogram czasów E2E oraz dopasowana krzywa Gaussa per model.	37
5.7	Kompromis Pareto: opóźnienie E2E a wskaźnik halucynacji.	38
5.8	Dekompozycja E2E na TTFT i <i>generation_ms</i> (cold cache).	41
5.9	Dekompozycja E2E na TTFT i <i>generation_ms</i> (warm cache).	42
5.10	Dopasowanie rozkładu log-normalnego do <i>generation_ms</i> (warm cache).	43
5.11	Dopasowanie rozkładu log-normalnego do <i>generation_ms</i> (cold cache).	43
5.12	Przyspieszenie <i>generation_ms</i> względem Q8_0 (warm cache).	44
5.13	Czas <i>generation_ms</i> według wariantu kwantyzacji.	45
5.14	Czas generacji a długość odpowiedzi.	46
5.15	Szybkość generacji według wariantu kwantyzacji.	46
6.1	Architektura prototypu Wilga.	52
6.2	Faza rozgrzewania modelu Ucznia.	54
6.3	Ekran startowy po rozgrzaniu modelu.	55
6.4	Stany Wilgusia w aplikacji użytkownika.	55
6.5	Widok aplikacji w trakcie prowadzenia rozmowy.	56

Spis tabel

4.1	Warianty rozmiaru modelu Whisper (OpenAI).	26
4.2	Rozwiązania TTS rozważane w projekcie.	26
4.3	Parametry generacji Ollama wykorzystywane w pracy.	27
5.1	Kategorie przypadków w złotym zbiorze testowym (20 przypadków).	31
5.2	Metryki oceny stosowane w benchmarku (trzy grupy).	31
5.3	Przestrzeń hiperparametrów przeszukiwana przez Optunę (TPE).	32
5.4	Modele kandydujące i ich udział w benchmarku Optuny.	33
5.5	Ranking modeli bez aktywnego chłodzenia.	33
5.6	Ranking modeli po zastosowaniu aktywnego chłodzenia.	34
5.7	Hiperparametry inferencji przyjęte do dalszych eksperymentów.	36
5.8	Czasy E2E w protokole Optuny (model Gaussa).	38
5.9	Rozmiary plików GGUF [GB] dla badanych modeli.	39
5.10	Wstępne porównanie wariantów kwantyzacji (E2E).	40
5.11	Wyniki kwantyzacji przy warm cache.	44

5.12 Struktura promptu Ucznia: prefiks, sufiks, typ i długość.	48
5.13 Wyniki fazy Optuny według modelu.	49
5.14 Porównanie czasów E2E dla cold i warm cache.	50
5.15 Rekomendowana konfiguracja produkcyjna Ucznia.	51

Spis załączników